บทที่ 3

การออกแบบและพัฒนาระบบ

3.1 ทฤษฎีที่เกี่ยวข้องกับระบบงาน

การพัฒนาระบบเว็บแอปพลิเคชันตรวจจับ Deepfake อัจฉริยะเพื่อความมั่นคงปลอดภัยสารสนเทศ (กรณีศึกษา: สำนักข่าว Lanner News) ใช้แนวทางการพัฒนาซอฟต์แวร์ตามวงจรชีวิตการพัฒนาระบบ (Software Development Life Cycle: SDLC) โดยประยุกต์ใช้รูปแบบ Iterative Waterfall Model ซึ่งเหมาะกับโครงงานที่มีขอบเขตงานชัดเจน แต่ยังต้องมีการทบทวนและปรับปรุงผลลัพธ์เป็นระยะในแต่ละช่วง แนวทางดังกล่าวช่วยให้ผู้พัฒนาสามารถวิเคราะห์ความต้องการ ออกแบบระบบ พัฒนา ทดสอบ และปรับปรุงระบบอย่างเป็นลำดับ พร้อมตรวจสอบความสอดคล้องของระบบกับผู้ใช้งานจริงได้ตลอดกระบวนการ

ในโครงงานนี้ได้แบ่งการดำเนินงานออกเป็น 6 ขั้นตอนหลัก ได้แก่

1) การรวบรวมและวิเคราะห์ความต้องการ (Requirement Analysis) รวบรวมความต้องการของผู้ใช้งาน วิเคราะห์ปัญหา ศึกษางานวิจัยที่เกี่ยวข้อง และจัดทำเอกสาร Business Requirement (BR) และ Software Requirement Specification (SRS)
2) การออกแบบระบบและฐานข้อมูล (System and Database Design) ออกแบบสถาปัตยกรรมระบบตามแนวคิด 3-Tier Architecture ออกแบบฐานข้อมูล (ER Diagram) กระบวนการทำงานของโมเดล Deep Learning และส่วนติดต่อผู้ใช้ (UI/UX)
3) การพัฒนาและฝึกโมเดล Deep Learning (Model Development) เตรียมและจัดการชุดข้อมูล FaceForensics++ C23 ฝึกสอนโมเดล XceptionNet ร่วมกับเทคนิค Context Expansion ประเมินประสิทธิภาพโมเดล และบันทึกโมเดลเพื่อนำไปใช้งานในระบบ
4) การพัฒนาระบบเว็บแอปพลิเคชัน (Web Application Development) พัฒนาส่วนติดต่อผู้ใช้ (Frontend) ด้วย Next.js พัฒนาส่วนบริการระบบ (Backend API) ด้วย FastAPI และเชื่อมต่อโมเดล AI เข้ากับระบบเว็บ
5) การทดสอบและประเมินผลระบบ (System Testing) ทดสอบการทำงานของระบบในระดับหน่วย (Unit Testing) และระดับบูรณาการ (Integration Testing) ทดสอบประสิทธิภาพการประมวลผลวิดีโอ และประเมินความพึงพอใจของผู้ใช้งาน (User Acceptance Testing: UAT)
6) การจัดทำเอกสารและนำเสนอ (Documentation and Presentation) จัดทำรายงานปริญญานิพนธ์ฉบับสมบูรณ์ คู่มือการใช้งาน และเตรียมการนำเสนอผลงานต่อคณะกรรมการ

3.2 แผนภาพกระแสข้อมูล (Data Flow Diagram)

แผนภาพกระแสข้อมูล (Data Flow Diagram: DFD) เป็นเครื่องมือที่ใช้ในการวิเคราะห์ออกแบบระบบสารสนเทศโดยสามารถแสดงให้เห็น ถึงการไหลของข้อมูลจากแหล่งกำเนิด (Source) ผ่านกระบวนการประมวลผล (Process) ไปยังจุดหมายปลายทาง (Destination) รวมถึงแหล่งจัดเก็บข้อมูล (Data Store) ที่เกี่ยวข้องในโครงงานนี้ได้จัดทำ DFD ไว้ 3 ระดับ ได้แก่ DFD Level 0(Context Diagram), DFD Level 1 และ DFD Level 2เพื่ออธิบายการทำงานของระบบอย่างละเอียดในแต่ละระดับ

3.2.1 สัญลักษณ์ที่ใช้ในแผนภาพกระแสข้อมูล (DFD Symbols)

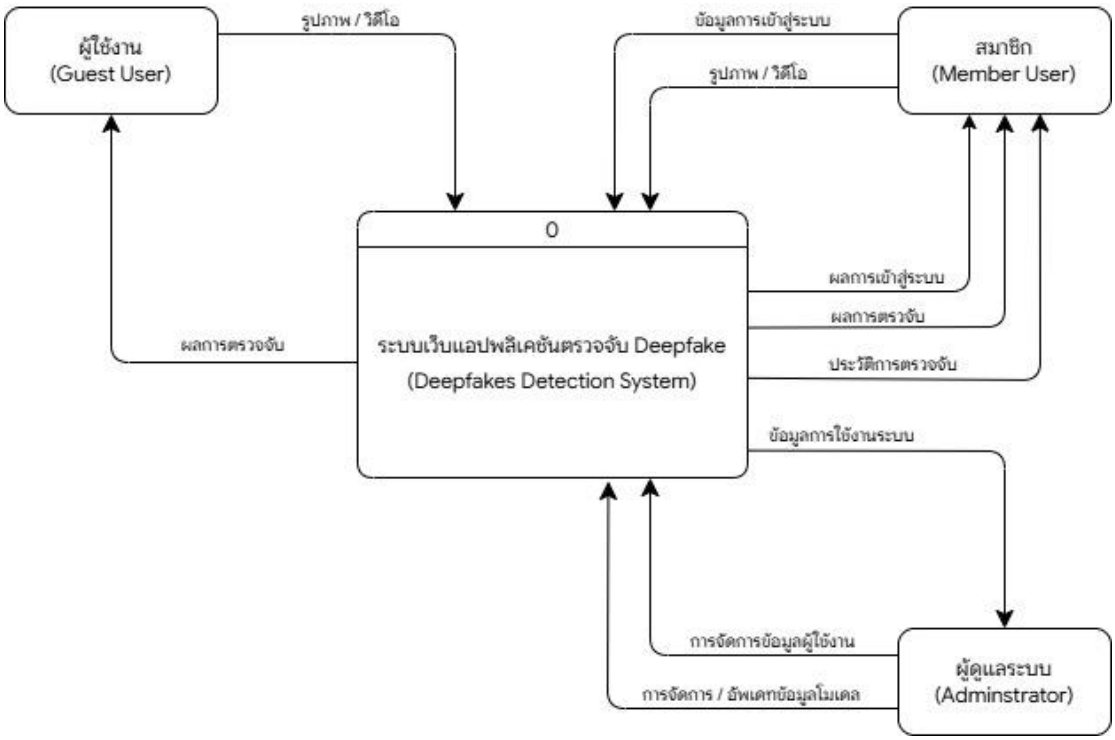
ในการเขียนแผนภาพกระแสข้อมูลของโครงงานนี้ ได้ใช้สัญลักษณ์ตามมาตรฐาน Gane & Sarson ซึ่งเป็นรูปแบบที่นิยมใช้ในการวิเคราะห์และออกแบบระบบ โดยมีสัญลักษณ์หลัก 4 ประเภท ดังตาราง

ตารางที่ 3.1 สัญลักษณ์ที่ใช้ในแผนภาพกระแสข้อมูล (DFD)

สัญลักษณ์	ชื่อ	ความหมาย
สี่เหลี่ยมมุมมน	Process	กระบวนการประมวลผลข้อมูลภายในระบบ ทำหน้าที่รับข้อมูลเข้า ประมวลผล และส่งผลลัพธ์ออก
สี่เหลี่ยมผืนผ้า	External Entity	ผู้ใช้หรือระบบภายนอกที่มีปฏิสัมพันธ์กับระบบ เป็นทั้งแหล่งส่งและรับข้อมูล
เส้นขนานเปิดด้านขวา	Data Store	แหล่งจัดเก็บข้อมูลที่ระบบใช้ในการอ่านหรือบันทึกข้อมูล เช่น ตารางในฐานข้อมูล PostgreSQL
เส้นลูกศร	Data Flow	ทิศทางการไหลของข้อมูลระหว่าง Process, External Entity และ Data Store

3.2.2 แผนภาพกระแสข้อมูลระดับบริบท (DFD Level 0 — Context Diagram)

แผนภาพบริบท (Context Diagram) หรือ DFD Level 0 เป็นแผนภาพระดับสูงสุดที่แสดงภาพรวมของระบบทั้งหมด ในรูปแบบกระบวนการเดียว (Process 0) โดยแสดงให้เห็นว่าระบบเว็บแอปพลิเคชันตรวจจับ Deepfake มีปฏิสัมพันธ์กับ External Entity ใดบ้างและมีข้อมูลใดที่ไหลเข้าและออกจากระบบ



รูปที่ 3.1 แผนภาพกระแสข้อมูลระดับบริบท (DFD Level 0) ระบบเว็บแอปพลิเคชันตรวจจับ Deepfake

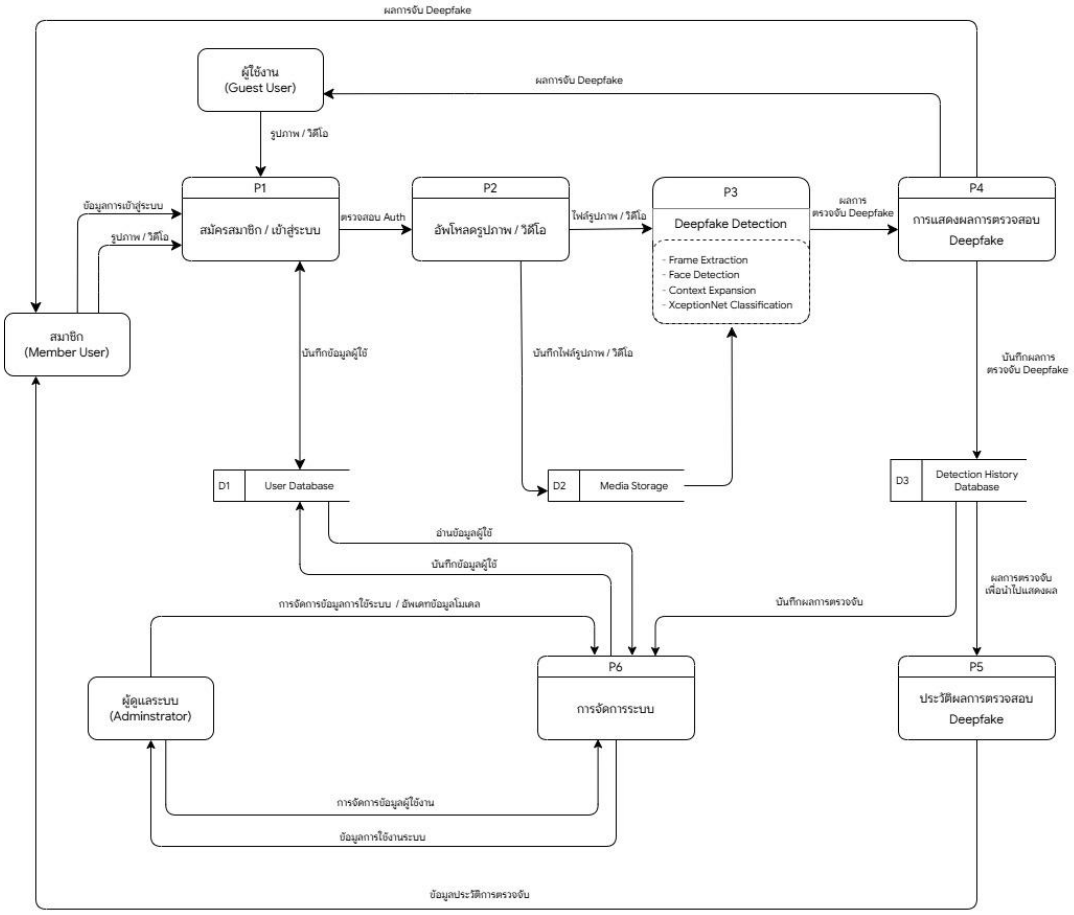
ตารางที่ 3.2 อธิบาย External Entity ของระบบเว็บแอปพลิเคชันตรวจจับ Deepfake

รหัส	External Entity	คำอธิบาย
E1	Guest User	ผู้ใช้งานที่ยังไม่ได้เข้าสู่ระบบ สามารถอัปโหลดรูปภาพหรือวิดีโอเพื่อให้ระบบตรวจจับ Deepfake และรับผลการวิเคราะห์ ได้แก่ ผลการตรวจสอบและค่า Confidence Score
E2	Member User	ผู้ใช้งานที่ลงทะเบียนและเข้าสู่ระบบ สามารถอัปโหลดรูปภาพหรือวิดีโอ ตรวจสอบผลการวิเคราะห์ ดูประวัติการตรวจสอบย้อนหลัง
E3	Administrator	ผู้ดูแลระบบ มีหน้าที่จัดการข้อมูลผู้ใช้งาน ตรวจสอบข้อมูลการใช้งาน จัดการข้อมูลที่เกี่ยวข้องกับระบบ และดูแลการทำงานของระบบให้มีความถูกต้องและปลอดภัย

3.2.3 แผนภาพกระแสข้อมูลระดับที่ 1 (DFD Level 1)

DFD Level 1 เป็นการขยายรายละเอียดจาก DFD Level 0 โดยแบ่งกระบวนการหลักของระบบออกเป็นกระบวนการย่อย เพื่อแสดงการไหลของข้อมูลระหว่างหน่วยงานภายนอก กระบวนการภายในระบบ และแหล่งจัดเก็บข้อมูล อย่างละเอียด

แผนภาพกระแสข้อมูลระดับที่ 1 ของระบบ เว็บแอปพลิเคชันตรวจจับ Deepfake อัจฉริยะเพื่อความมั่นคงปลอดภัยสารสนเทศ แสดงกระบวนการทำงานหลัก 6 กระบวนการ ได้แก่ (1) สมัครสมาชิก/เข้าสู่ระบบ รับข้อมูลการสมัครสมาชิกและการเข้าสู่ระบบจากผู้ใช้งาน ทำการตรวจสอบข้อมูลผู้ใช้ก่อนอนุญาตให้เข้าสู่ระบบ พร้อมจัดเก็บข้อมูลบัญชีผู้ใช้ลงในฐานข้อมูล (2) อัปโหลดรูปภาพหรือวิดีโอ รับไฟล์รูปภาพหรือวิดีโอจาก Guest User หรือ Member User ตรวจสอบชนิดและขนาดของไฟล์ จากนั้นจัดเก็บไฟล์และส่งต่อไปยังกระบวนการตรวจจับ Deepfake (3) ตรวจจับ Deepfake ดำเนินการประมวลผลข้อมูลที่ได้รับ โดยแยกเฟรมจากวิดีโอ ตรวจจับใบหน้า ขยายบริบทของภาพ และวิเคราะห์ด้วยโมเดล XceptionNet เพื่อจำแนกผลเป็น Real หรือ Fake พร้อมคำนวณค่า Confidence Score (4) แสดงผลการตรวจจับ รับผลลัพธ์จากกระบวนการตรวจจับ Deepfake แล้วแสดงผลการวิเคราะห์แก่ผู้ใช้งาน ประกอบด้วยผลการจำแนก (Real/Fake) ค่า Confidence Score และรายละเอียดของการตรวจจับ (5) จัดเก็บและเรียกดูประวัติการตรวจจับ จัดเก็บผลการตรวจจับของ Member User ลงในฐานข้อมูล เพื่อให้สามารถเรียกดูผลการตรวจสอบย้อนหลังได้ โดย Guest User จะไม่สามารถใช้งานส่วนนี้ได้ (6) จัดการระบบ ผู้ดูแลระบบสามารถจัดการข้อมูลผู้ใช้งาน ตรวจสอบประวัติการตรวจจับ ดูข้อมูลการใช้งาน และบริหารจัดการข้อมูลภายในระบบ เพื่อให้ระบบสามารถทำงานได้อย่างถูกต้องและมีประสิทธิภาพ



รูปที่ 3.2 แผนภาพกระแสข้อมูลระดับที่ 1 (DFD Level 1) ระบบเว็บแอปพลิเคชันตรวจจับ Deepfake

ตารางที่ 3.3 อธิบาย Process ใน DFD Level 1

รหัส	External Entity	คำอธิบาย
P1	สมัครสมาชิก / เข้าสู่ระบบ	รับข้อมูลการสมัครสมาชิกและเข้าสู่ระบบ ตรวจสอบข้อมูลผู้ใช้ และบันทึกข้อมูลลงในฐานข้อมูลผู้ใช้
P2	อัปโหลดรูปภาพ / วิดีโอ	รับไฟล์จากผู้ใช้งาน ตรวจสอบประเภทของไฟล์ และจัดเก็บไฟล์ก่อนส่งเข้าสู่การวิเคราะห์
P3	ตรวจจับ Deepfake	วิเคราะห์รูปภาพหรือวิดีโอด้วย Frame Extraction, Face Detection, Context Expansion และโมเดล XceptionNet เพื่อจำแนกผลและคำนวณค่า Confidence Score
P4	แสดงผลการตรวจจับ	แสดงผลการตรวจจับและค่า Confidence Score พร้อมบันทึกผลการวิเคราะห์
P5	ประวัติผลการตรวจจับ	จัดเก็บและเรียกดูประวัติผลการตรวจจับย้อนหลังของสมาชิก
P6	การจัดการระบบ	จัดการข้อมูลผู้ใช้งาน ตรวจสอบข้อมูลการใช้งาน และบริหารจัดการข้อมูลภายในระบบ

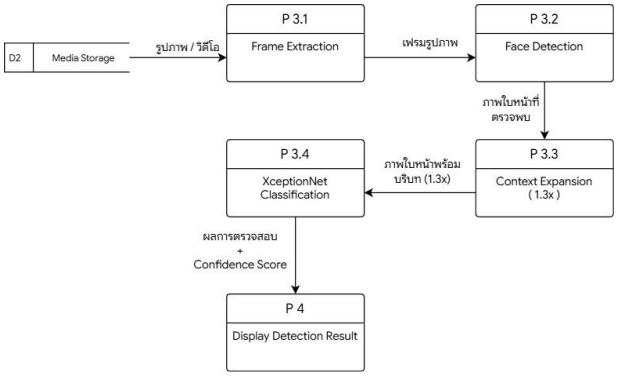
ตารางที่ 3.4 อธิบาย Data Store ใน DFD Level 1

รหัส	กระบวนการ	คำอธิบาย
D1	User Database	จัดเก็บข้อมูลบัญชีผู้ใช้ เช่น ชื่อผู้ใช้ อีเมล รหัสผ่านที่เข้ารหัส และสิทธิ์การใช้งาน
D2	Media Storage	จัดเก็บไฟล์รูปภาพและวิดีโอที่ผู้ใช้อัปโหลดเพื่อใช้ในการตรวจจับ Deepfake
D3	Detection History Database	จัดเก็บผลการตรวจจับ ประวัติการตรวจสอบ ค่า Confidence Score วันที่และเวลาที่ตรวจสอบ เพื่อรองรับการเรียกดูย้อนหลัง

3.2.4 แผนภาพกระแสข้อมูลระดับที่ 2 (DFD Level 2) กระบวนการ P3 Deepfake Detection

DFD Level 2 เป็นการขยายรายละเอียดของกระบวนการ P3 Deepfake Detection จาก DFD Level 1 เพื่อแสดงลำดับขั้นตอนการประมวลผลของระบบตรวจจับสื่อปลอม (Deepfake) ตั้งแต่การรับข้อมูลภาพหรือวิดีโอ การเตรียมข้อมูล การตรวจจับใบหน้า การขยายบริบทของภาพ จนถึงการจำแนกผลด้วยแบบจำลอง XceptionNet โดยแบ่งออกเป็น 4 กระบวนการย่อย ดังนี้

กระบวนการเริ่มต้นจาก **P3.1 Frame Extraction** โดยระบบรับข้อมูลภาพหรือวิดีโอจาก D2 Media Storage หากเป็นไฟล์วิดีโอ ระบบจะทำการแยกวิดีโอออกเป็นเฟรมภาพ เพื่อให้สามารถประมวลผลแต่ละเฟรมได้อย่างมีประสิทธิภาพ ส่วนกรณีเป็นไฟล์ภาพจะส่งภาพเข้าสู่ขั้นตอนถัดไปโดยตรงจากนั้นข้อมูลจะถูกส่งเข้าสู่ **P3.2 Face Detection** ซึ่งทำหน้าที่ตรวจจับตำแหน่งใบหน้าในแต่ละเฟรมหรือภาพด้วยอัลกอริทึมตรวจจับใบหน้า เพื่อเลือกเฉพาะบริเวณที่เป็นใบหน้าสำหรับการวิเคราะห์ ลดข้อมูลที่ไม่จำเป็นและเพิ่มความแม่นยำของแบบจำลอง เมื่อได้ตำแหน่งใบหน้าแล้ว ระบบจะดำเนินการใน **P3.3 Context Expansion (35%)** โดยขยายพื้นที่รอบใบหน้าเพิ่มอีก 35% เพื่อเก็บบริบทโดยรอบของใบหน้า เช่น ขอบใบหน้า เส้นผม และบริเวณใกล้เคียง ซึ่งช่วยให้แบบจำลองสามารถเรียนรู้ลักษณะของภาพได้ครบถ้วนมากยิ่งขึ้นและช่วยเพิ่มประสิทธิภาพในการตรวจจับ Deepfake ขั้นตอนสุดท้ายคือ **P3.4 XceptionNet Classification** ซึ่งนำภาพใบหน้าที่ผ่านการขยายบริบทเข้าสู่แบบจำลอง XceptionNet เพื่อจำแนกว่าภาพหรือวิดีโอเป็น Real หรือ Fake พร้อมคำนวณค่าความเชื่อมั่น (Confidence Score) จากนั้นส่งผลการตรวจจับ (Detection Result) ไปยัง P4 การแสดงผลการตรวจสอบ Deepfake เพื่อแสดงผลให้แก่ผู้ใช้งาน และบันทึกข้อมูลการตรวจสอบลงในฐานข้อมูลประวัติการตรวจสอบของระบบตามลำดับ



รูปที่ 3.3 แผนภาพกระแสข้อมูลระดับที่ 2 (DFD Level 2) ของกระบวนการ P3 Deepfake Detection

ตารางที่ 3.5 อธิบาย Sub-Process ใน DFD Level 2 ของกระบวนการ P3 Deepfake Detection

รหัส	กระบวนการ	คำอธิบาย
P3.1	Frame Extraction	รับไฟล์ภาพหรือวิดีโอจาก D2 Media Storage โดยระบบจะแยกวิดีโอออกเป็นเฟรมภาพ หรือส่งไฟล์ภาพเข้าสู่ขั้นตอนถัดไปเพื่อเตรียมข้อมูลสำหรับการวิเคราะห์
P3.2	Face Detection	ตรวจจับตำแหน่งใบหน้าในแต่ละเฟรมหรือภาพ และคัดเลือกเฉพาะบริเวณใบหน้าที่ใช้ในการวิเคราะห์ เพื่อลดข้อมูลที่ไม่เกี่ยวข้อง
P3.3	Context Expansion (35%)	ขยายพื้นที่รอบใบหน้าเพิ่มอีก 35% เพื่อเก็บข้อมูลบริบทโดยรอบและเพิ่มประสิทธิภาพในการตรวจจับ Deepfake
P3.4	Deepfake Classification (XceptionNet)	วิเคราะห์ภาพใบหน้าด้วยแบบจำลอง XceptionNet เพื่อจำแนกผลเป็น Real หรือ Fake พร้อมคำนวณค่า Confidence Score และส่งผลการตรวจจับไปยังกระบวนการแสดงผล

3.2.5 สถาปัตยกรรมระบบ (System Architecture)

ระบบเว็บแอปพลิเคชันตรวจจับ Deepfake อัจฉริยะเพื่อความมั่นคงปลอดภัยสารสนเทศ พัฒนาในรูปแบบ Web Application รองรับการใช้งานผ่านเว็บเบราว์เซอร์บนคอมพิวเตอร์และอุปกรณ์เคลื่อนที่ (Responsive Web Design) โดยใช้สถาปัตยกรรมแบบ 3-Tier Architecture ประกอบด้วย Presentation Layer, Application Layer และ Data Layer เพื่อแยกหน้าที่การทำงานของแต่ละส่วนออกจากกันอย่างชัดเจน ส่งผลให้ระบบมีความยืดหยุ่น สามารถบำรุงรักษา พัฒนาเพิ่มเติม และขยายระบบในอนาคตได้อย่างมีประสิทธิภาพ

สถาปัตยกรรมของระบบแบ่งออกเป็น 3 ชั้นหลัก ได้แก่ Presentation Layer (Frontend), Application Layer (Backend และ Deep Learning Model) และ Data Layer (Database และ Cloud Storage) โดยแต่ละชั้นทำหน้าที่แตกต่างกันและติดต่อสื่อสารผ่านรูปแบบ RESTful API ซึ่งช่วยลดการพึ่งพากันระหว่างแต่ละส่วนของระบบและเพิ่มความสามารถในการดูแลรักษาระบบ

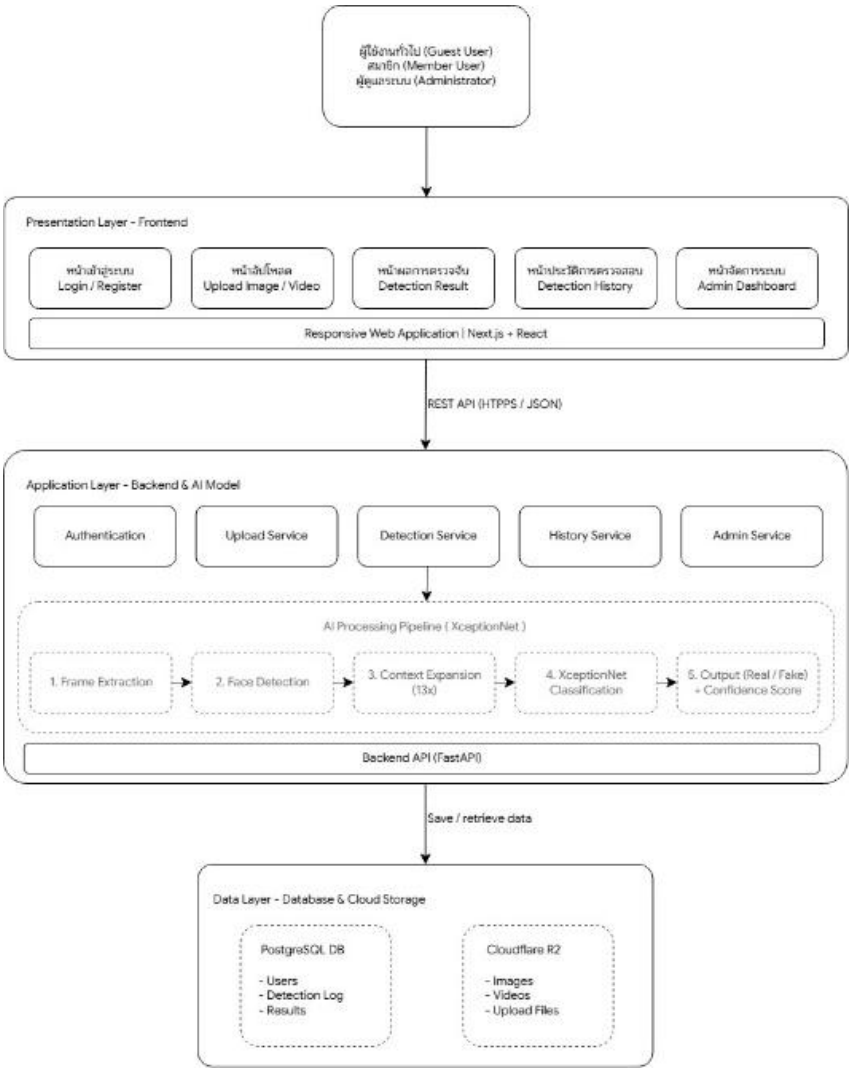
ในส่วนของ Presentation Layer (Next.js Frontend) เป็นส่วนที่ผู้ใช้งานโต้ตอบกับระบบโดยตรง พัฒนาด้วย Next.js และ React ประกอบด้วยหน้าหลักของระบบ เช่น หน้าเข้าสู่ระบบ, หน้าอัปโหลดรูปภาพหรือวิดีโอ, หน้าผลการตรวจสอบ, หน้าประวัติการตรวจสอบ และหน้าจัดการระบบ โดย Frontend ทำหน้าที่รับข้อมูลจากผู้ใช้งาน ส่งคำร้องขอ ไปยัง Backend และแสดงผลการตรวจจับ Deepfake พร้อมค่าความเชื่อมั่น ให้แก่ผู้ใช้งาน

ถัดมาเป็น Application Layer (FastAPI และ Deep Learning Model) ซึ่งทำหน้าที่เป็นศูนย์กลางในการประมวลผลและควบคุมตรรกะการทำงานของระบบ พัฒนาด้วย FastAPI ทำหน้าที่รับคำร้องจาก Frontend ตรวจสอบข้อมูลผู้ใช้งาน จัดการไฟล์ที่อัปโหลด และเชื่อมต่อกับแบบจำลอง XceptionNet สำหรับการตรวจจับ Deepfake โดยกระบวนการตรวจจับประกอบด้วยการแยกเฟรมจากวิดีโอ การตรวจจับใบหน้า การขยายบริบทของใบหน้า และการจำแนกภาพด้วยแบบจำลอง XceptionNet เพื่อให้ผลลัพธ์เป็นการจำแนก Real หรือ Fake พร้อมค่าความเชื่อมั่น

ส่วน Data Layer (PostgreSQL และ Cloudflare R2) ทำหน้าที่จัดเก็บข้อมูลทั้งหมดของระบบโดยใช้ PostgreSQL สำหรับจัดเก็บข้อมูลผู้ใช้งาน ข้อมูลประวัติการตรวจสอบ และข้อมูลที่เกี่ยวข้องกับระบบ ส่วนไฟล์รูปภาพและวิดีโอที่ผู้ใช้อัปโหลดจะถูกจัดเก็บไว้ใน Cloudflare R2 Object Storage เพื่อรองรับการจัดเก็บไฟล์

ขนาดใหญ่ ลดภาระของฐานข้อมูล และเพิ่มประสิทธิภาพในการเข้าถึงข้อมูล นอกจากนี้ระบบยังใช้ฐานข้อมูลในการจัดเก็บผลการตรวจจับและประวัติการใช้งาน เพื่อให้ผู้ใช้งานสามารถเรียกดูข้อมูลย้อนหลังได้

การทำงานของทั้ง 3 ชั้นเชื่อมต่อกันผ่านรูปแบบ Request–Response โดยผู้ใช้งานส่งข้อมูลรูปภาพหรือวิดีโอผ่าน Frontend ไปยัง Backend จากนั้น Backend จะดำเนินการประมวลผลร่วมกับแบบจำลอง XceptionNet และจัดเก็บข้อมูลที่เกี่ยวข้องลงใน PostgreSQL และ Cloudflare R2 ก่อนส่งผลการตรวจจับพร้อมค่าความเชื่อมั่นกลับมายัง Frontend เพื่อแสดงผลให้แก่ผู้ใช้งาน ทำให้ระบบสามารถรองรับการตรวจจับ Deepfake การจัดเก็บข้อมูล และการบริหารจัดการระบบได้อย่างมีประสิทธิภาพ



รูปที่ 3.4 สถาปัตยกรรมระบบแบบ 3 ชั้น (3-Tier Architecture) ของระบบเว็บแอปพลิเคชันตรวจจับ Deepfake

3.3 เทคโนโลยีที่ใช้ในการพัฒนาระบบ (Technology Stack)

โครงงานนี้เลือกใช้เทคโนโลยีที่เหมาะสมสำหรับการพัฒนาเว็บแอปพลิเคชันตรวจจับ Deepfake โดยมุ่งเน้นการประยุกต์ใช้เทคโนโลยีการเรียนรู้เชิงลึก (Deep Learning) สำหรับการจำแนกภาพและวิดีโอ ร่วมกับการพัฒนาเว็บแอปพลิเคชันแบบตอบสนอง และการจัดเก็บข้อมูลบนระบบคลาวด์ (Cloud-based Storage) เพื่อให้

ระบบสามารถตรวจจับสื่อปลอมได้อย่างมีประสิทธิภาพ รองรับการใช้งานผ่านเว็บเบราว์เซอร์ และสามารถขยายระบบในอนาคตได้ง่าย

ตารางที่ 3.6 เทคโนโลยีที่ใช้ในการพัฒนาระบบเว็บแอปพลิเคชันตรวจจับ Deepfake

หมวดเทคโนโลยี	เทคโนโลยีที่ใช้	วัตถุประสงค์
Frontend	Next.js (React)	พัฒนาส่วนติดต่อผู้ใช้งานของเว็บแอปพลิเคชัน
Backend	FastAPI	พัฒนา RESTful API และควบคุมการทำงานของระบบ
Deep Learning	PyTorch, XceptionNet	พัฒนา Backend และกระบวนการประมวลผลของแบบจำลอง
Computer Vision	OpenCV, RetinaFace	พัฒนา ฝึก และใช้งานแบบจำลอง Deep Learning
Image Processing	Context Expansion (35%)	ขยายบริเวณรอบใบหน้าเพื่อเพิ่มบริบทก่อนการจำแนก
Database	PostgreSQL	จัดเก็บข้อมูลผู้ใช้งานและประวัติการตรวจสอบ
Cloud Storage	Cloudflare R2	จัดเก็บไฟล์ภาพและวิดีโอที่ผู้ใช้อัปโหลด
Authentication	JSON Web Token (JWT)	ยืนยันตัวตนและกำหนดสิทธิ์การเข้าใช้งานของผู้ใช้
Model Development	Kaggle, Google Colab	ใช้จัดเก็บชุดข้อมูลและฝึกแบบจำลอง Deep Learning บนระบบ Cloud
Development Tools	Visual Studio Code	ใช้พัฒนาและแก้ไขซอร์สโค้ดของระบบ
Deployment	Docker และ Uvicorn	ใช้สำหรับติดตั้งและให้บริการระบบ

3.4 การออกแบบโมเดล Deep Learning (Deep Learning Model Design)

การออกแบบโมเดล Deep Learning เป็นขั้นตอนสำคัญของโครงงานนี้ ซึ่งครอบคลุมตั้งแต่การเตรียมชุดข้อมูลไปจนถึงการกำหนดวิธีการที่ใช้ตรวจจับ Deepfake โดยมีวัตถุประสงค์เพื่อให้ได้แบบจำลองที่สามารถจำแนกวิดีโอและภาพว่าเป็นสื่อจริงหรือสื่อปลอมได้อย่างแม่นยำ แม้ในกรณีที่วิดีโอผ่านการบีบอัดจากการเผยแพร่บนสื่อออนไลน์ ในหัวข้อนี้จะอธิบายรายละเอียดตั้งแต่ชุดข้อมูลที่ใช้ เทคนิคการเพิ่มความหลากหลายของข้อมูล กระบวนการเตรียมข้อมูลและการแบ่งชุดข้อมูล ไปจนถึงวิธีการที่ผู้พัฒนานำเสนอ ได้แก่ เทคนิคการขยายบริบทของใบหน้า (Context Expansion) ร่วมกับสถาปัตยกรรมแบบจำลอง XceptionNet การกำหนดค่าการฝึกสอน ตัวชี้วัดที่ใช้ประเมินผล และเทคนิค Grad-CAM ที่นำมาใช้เพื่อเพิ่มความโปร่งใสในการอธิบายผลการตัดสินใจของแบบจำลอง

3.4.1 ชุดข้อมูล (Dataset)

โครงงานนี้ใช้ชุดข้อมูล FaceForensics++ (FF++) ซึ่งเป็นชุดข้อมูลมาตรฐานที่ได้รับความนิยมอย่างแพร่หลายในงานวิจัยด้านการตรวจจับสื่อปลอม เนื่องจากประกอบด้วยข้อมูลวิดีโอจริง และวิดีโอปลอม ที่สร้างขึ้นจากหลายเทคนิคการสร้าง Deepfake ทำให้สามารถใช้สำหรับการฝึกและประเมินประสิทธิภาพของแบบจำลองการเรียนรู้เชิงลึกได้อย่างครอบคลุม ในการศึกษาครั้งนี้ ผู้พัฒนาเลือกใช้ชุดข้อมูล FaceForensics++ ระดับการบีบอัด C23 (Compression Level 23) ซึ่งเป็นระดับการบีบอัดแบบปานกลาง โดยใช้การเข้ารหัสวิดีโอแบบ H.264 ที่ค่า Constant Rate Factor (CRF) เท่ากับ 23 เพื่อจำลองสภาพของวิดีโอที่เผยแพร่ผ่านแพลตฟอร์มออนไลน์ได้ใกล้เคียงกับการใช้งานจริง ชุดข้อมูลประกอบด้วยวิดีโอทั้งหมด 7,000 วิดีโอ แบ่งเป็นวิดีโอจริง (Real) จำนวน 1,000 วิดีโอ และวิดีโอปลอม (Fake) จำนวน 6,000 วิดีโอ ซึ่งสร้างจากเทคนิคการสร้าง Deepfake จำนวน 6 ประเภท ได้แก่ DeepFakeDetection, Deepfakes, Face2Face, FaceShifter, FaceSwap และ NeuralTextures นอกจากนี้ยังมีไฟล์ Metadata ในรูปแบบ CSV จำนวน 10 ไฟล์ สำหรับใช้อ้างอิงรายละเอียดของข้อมูลแต่ละรายการ ก่อนนำข้อมูลเข้าสู่แบบจำลอง ระบบจะดำเนินการแยกเฟรมจากวิดีโอ ตรวจจับใบหน้า และขยายบริบทของใบหน้า เพิ่มอีก 35% เพื่อเตรียมข้อมูลก่อนนำเข้าสู่แบบจำลอง XceptionNet สำหรับการจำแนกผลว่าเป็นวิดีโอจริง หรือวิดีโอปลอม

ตารางที่ 3.7 รายละเอียดชุดข้อมูล FaceForensics++ ที่ใช้ในการพัฒนาแบบจำลอง

ลำดับ	ชุดข้อมูล	จำนวนวิดีโอ	ประเภท
1	Original	1,000	Real
2	DeepFakeDetection	1,000	Fake
3	Deepfakes	1,000	Fake
4	Face2Face	1,000	Fake
5	FaceShifter	1,000	Fake
6	FaceSwap	1,000	Fake
7	NeuralTextures	1,000	Fake

เพื่อให้แบบจำลองสามารถเรียนรู้และประเมินผลได้อย่างมีประสิทธิภาพ ชุดข้อมูลถูกแบ่งออกเป็น 3 ส่วน ได้แก่ Training Set, Validation Set และ Testing Set โดยใช้อัตราส่วน 70 : 15 : 15 ซึ่งเป็นอัตราส่วนที่นิยมใช้ในการพัฒนาแบบจำลองการเรียนรู้เชิงลึก เนื่องจากช่วยให้มีข้อมูลเพียงพอสำหรับการฝึกแบบจำลอง การปรับแต่งค่าพารามิเตอร์ และการประเมินผลด้วยข้อมูลที่ไม่เคยใช้ในการฝึกมาก่อน

3.4.2 การเพิ่มความหลากหลายของข้อมูล (Data Augmentation)

เพื่อเพิ่มความสามารถในการเรียนรู้ของแบบจำลองและลดปัญหาการเกิด Overfitting ผู้วิจัยได้นำเทคนิค Data Augmentation มาใช้กับชุดข้อมูลสำหรับการฝึก โดยทำการแปลงภาพในรูปแบบต่าง ๆ แบบสุ่ม เพื่อจำลองความหลากหลายของข้อมูลที่อาจเกิดขึ้นในการใช้งานจริง เช่น การเปลี่ยนมุมมองของใบหน้า การเปลี่ยนแปลงสภาพแสง และการลดคุณภาพของภาพจากการบีบอัดไฟล์ เทคนิค Data Augmentation ถูกนำมาใช้เฉพาะกับ Training Set ส่วน Validation Set จะใช้เฉพาะการปรับขนาดภาพและการทำ Normalization เพื่อให้ผลการประเมินสะท้อนประสิทธิภาพของแบบจำลองได้อย่างถูกต้องและไม่เกิดความเอนเอียงจากการแปลงข้อมูล

ตารางที่ 3.8 รายละเอียดเทคนิค Data Augmentation ที่ใช้ในการฝึกแบบจำลอง

ลำดับ	เทคนิค	คำอธิบาย
1	Resize	ปรับขนาดภาพเป็น 320 × 320 พิกเซล ก่อนเข้าสู่กระบวนการสุ่มตัดภาพ
2	Random Resized Crop	สุ่มตัดและปรับขนาดภาพเป็น 299 × 299 พิกเซล โดยกำหนด Scale 0.80–1.00 และ Ratio 0.95–1.05
3	Random Horizontal Flip	พลิกภาพในแนวนอนแบบสุ่ม เพื่อเพิ่มความหลากหลายของข้อมูล
4	Color Jitter	ปรับค่าความสว่าง, ความเปรียบต่าง, ความอิ่มตัวของสี และเฉดสีแบบสุ่ม
5	Gaussian Blur	เพิ่มความเบลอของภาพแบบสุ่มด้วย Kernel Size เท่ากับ 3 และความน่าจะเป็น 30%
6	Random JPEG Compression	จำลองการบีบอัดไฟล์ JPEG ด้วยคุณภาพระหว่าง 40–95 และความน่าจะเป็น 50% เพื่อเลียนแบบภาพที่ถูกบีบอัดจากสื่อออนไลน์
7	ToTensor	แปลงข้อมูลภาพให้อยู่ในรูปแบบ Tensor
8	Normalization	ปรับค่าพิกเซลให้อยู่ในช่วงมาตรฐานด้วย Mean = (0.5, 0.5, 0.5) และ Standard Deviation = (0.5, 0.5, 0.5)

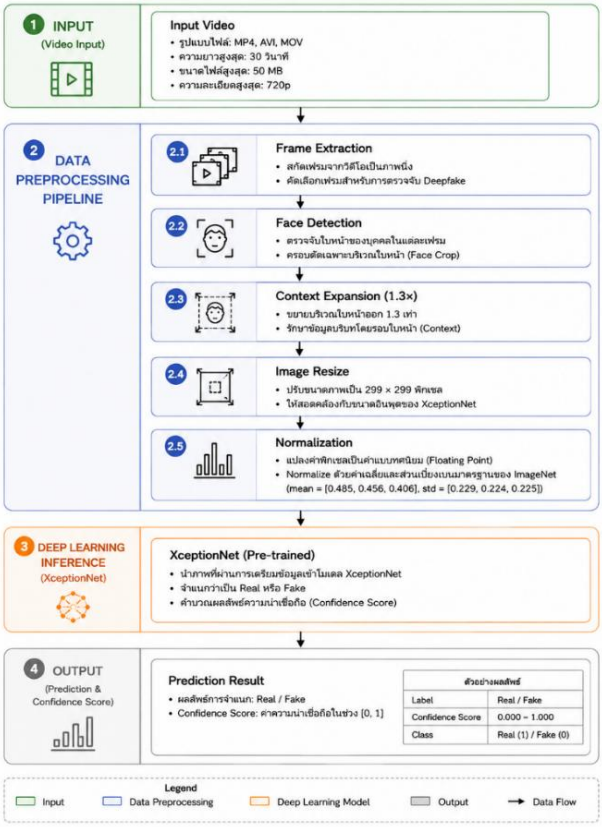
3.4.3 การเตรียมข้อมูล (Data Preprocessing)

การเตรียมข้อมูล (Data Preprocessing) เป็นขั้นตอนสำคัญก่อนนำข้อมูลเข้าสู่กระบวนการฝึกแบบจำลอง โดยมีวัตถุประสงค์เพื่อปรับปรุงคุณภาพของข้อมูล ลดข้อมูลที่ไม่เกี่ยวข้อง และเตรียมข้อมูลให้อยู่ในรูปแบบที่เหมาะสมสำหรับการเรียนรู้ของแบบจำลอง XceptionNet ซึ่งช่วยเพิ่มประสิทธิภาพในการตรวจจับภาพปลอมแปลง และลดความคลาดเคลื่อนที่อาจเกิดขึ้นจากข้อมูลต้นฉบับ ในงานวิจัยนี้ กระบวนการเตรียมข้อมูลประกอบด้วย 5 ขั้นตอนหลัก ได้แก่ การสกัดเฟรมจากวิดีโอ การตรวจจับใบหน้า การขยายบริบทของใบหน้า การปรับขนาดภาพ และการปรับค่าพิกเซลของภาพ

ตารางที่ 3.9 รายละเอียดขั้นตอนการเตรียมข้อมูล (Data Preprocessing)

ลำดับ	เทคนิค	คำอธิบาย
1	Frame Extraction	สกัดภาพนิ่ง Frame จากวิดีโอเพื่อใช้เป็นข้อมูลสำหรับการฝึกแบบจำลอง
2	Face Detection	ตรวจจับและตัดเฉพาะบริเวณใบหน้าจากแต่ละเฟรม เพื่อลดข้อมูลพื้นหลังที่ไม่เกี่ยวข้อง
3	Context Expansion	ขยายพื้นที่รอบใบหน้าเพิ่มอีก 35% เพื่อเพิ่มข้อมูลบริบท เช่น เส้นผม ลำคอ และขอบใบหน้า
4	Image Resizing	ปรับขนาดภาพเป็น 299 × 299 พิกเซล ให้สอดคล้องกับขนาดข้อมูลนำเข้าของแบบจำลอง XceptionNet
5	Normalization	ปรับค่าพิกเซลของภาพให้อยู่ในช่วงมาตรฐาน โดยใช้ค่า Mean = (0.5, 0.5, 0.5) และ Standard Deviation = (0.5, 0.5, 0.5)

(1) การสกัดเฟรม ทำการสกัดภาพนิ่ง จากวิดีโอทุกรายการในอัตรา 1 เฟรมต่อวินาที (1 fps) เนื่องจากวิดีโอ 1 คลิปประกอบด้วยเฟรมจำนวนมากที่มีความคล้ายคลึงกันสูงในช่วงเวลาที่ใกล้กัน (Temporal Redundancy) การสุ่มเฟรมในอัตรานี้จึงช่วยให้ได้ภาพนิ่งที่มีความหลากหลายเพียงพอสำหรับการฝึกแบบจำลอง โดยไม่เพิ่มภาระในการประมวลผลจากข้อมูลที่ซ้ำซ้อนโดยไม่จำเป็น (2) การตรวจจับใบหน้า ใช้ RetinaFace ในการตรวจจับตำแหน่งใบหน้าในแต่ละเฟรม และตัดเฉพาะบริเวณใบหน้าออกมา เพื่อลดข้อมูลพื้นหลังที่ไม่เกี่ยวข้องกับการจำแนก Deepfake โดยกรองเฉพาะเฟรมที่ตรวจพบใบหน้าชัดเจนและมีขนาดไม่น้อยกว่า 64×64 พิกเซล เฟรมที่ไม่พบใบหน้าหรือใบหน้ามีขนาดเล็กเกินไปจะถูกคัดออกจากชุดข้อมูล เพื่อรักษาคุณภาพของข้อมูลที่จะนำไปฝึกแบบจำลอง (3) การขยายบริบทของใบหน้า ขยายพื้นที่รอบใบหน้าที่ตรวจจับได้เพิ่มอีก 35% ในทุกทิศทาง เพื่อเพิ่มข้อมูลบริบทโดยรอบใบหน้า เช่น เส้นผม ลำคอ และขอบใบหน้า เข้าไปในภาพที่จะนำไปฝึกแบบจำลองด้วย เนื่องจากบริเวณดังกล่าวมักเป็นจุดที่กระบวนการสร้าง Deepfake ทิ้งร่องรอยความผิดปกติไว้ เช่น รอยต่อของสีผิว หรือขอบภาพที่เบลอผิดธรรมชาติ การขยายบริบทจึงช่วยให้แบบจำลองมีข้อมูลเพียงพอสำหรับเรียนรู้ลักษณะความผิดปกติเหล่านี้ (4) การปรับขนาดภาพ ปรับขนาดภาพใบหน้าที่ผ่านการขยายบริบทแล้วให้เป็น 299 × 299 พิกเซล ซึ่งเป็นขนาดข้อมูลนำเข้ามาตรฐานของแบบจำลอง XceptionNet เพื่อให้ภาพทุกภาพในชุดข้อมูลมีขนาดสม่ำเสมอและสอดคล้องกับโครงสร้างของแบบจำลองที่ผ่านการ Pre-trained มา (5) การปรับค่าพิกเซลของภาพ ปรับค่าพิกเซลของภาพให้อยู่ในช่วงมาตรฐาน โดยใช้ค่าเฉลี่ย (Mean) เท่ากับ (0.5, 0.5, 0.5) และส่วนเบี่ยงเบนมาตรฐาน (Standard Deviation) เท่ากับ (0.5, 0.5, 0.5) เพื่อให้การกระจายของค่าพิกเซลสอดคล้องกับน้ำหนักของแบบจำลอง Pre-trained ซึ่งช่วยให้แบบจำลองเรียนรู้และลู่เข้าสู่คำตอบ ได้เร็วและเสถียรขึ้นระหว่างการฝึกสอน



รูปที่ 3.5 กระบวนการเตรียมข้อมูลก่อนการฝึกแบบจำลอง (Data Preprocessing Pipeline)

3.4.4 การแบ่งชุดข้อมูล (Dataset Splitting)

การแบ่งชุดข้อมูล (Dataset Splitting) เป็นขั้นตอนสำคัญที่ช่วยให้สามารถประเมินประสิทธิภาพของแบบจำลองได้อย่างน่าเชื่อถือ โดยมีวัตถุประสงค์เพื่อแบ่งข้อมูลออกเป็นส่วนที่ใช้สำหรับการฝึกแบบจำลอง การปรับแต่งค่าพารามิเตอร์ระหว่างการฝึก และการประเมินผลด้วยข้อมูลที่แบบจำลองไม่เคยพบมาก่อน ซึ่งช่วยลดความเสี่ยงของการ Overfitting และทำให้ผลการประเมินสะท้อนความสามารถของแบบจำลองในการใช้งานจริงได้อย่างแม่นยำ ในงานวิจัยนี้ ชุดข้อมูลถูกแบ่งออกเป็น 3 ส่วน ได้แก่ Training Set, Validation Set และ Testing Set โดยใช้อัตราส่วน 70 : 15 : 15

ตารางที่ 3.10 รายละเอียดการแบ่งชุดข้อมูลสำหรับการพัฒนาแบบจำลอง

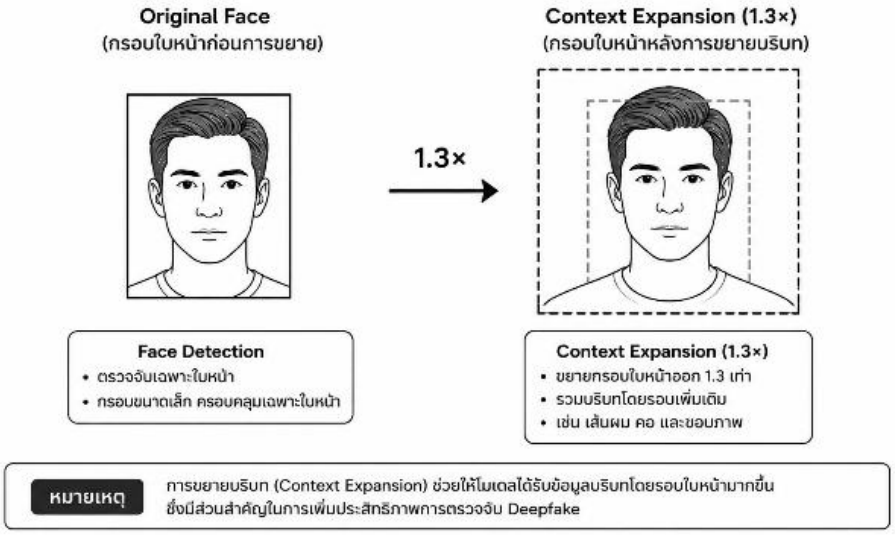
ชุดข้อมูล	อัตราส่วน (%)	คำอธิบาย
Training Set	70%	ใช้ฝึกแบบจำลอง XceptionNet ให้เรียนรู้ลักษณะของข้อมูล
Validation Set	15%	ใช้ตรวจสอบประสิทธิภาพระหว่างการฝึกและปรับแต่งค่าพารามิเตอร์ของแบบจำลอง
Testing Set	15%	ใช้ประเมินประสิทธิภาพของแบบจำลองด้วยข้อมูลที่ไม่เคยใช้ในการฝึก

(1) Training Set คือชุดข้อมูลส่วนใหญ่ที่คิดเป็นร้อยละ 70 ของข้อมูลทั้งหมด ใช้สำหรับฝึกให้แบบจำลอง XceptionNet เรียนรู้ลักษณะเฉพาะที่แตกต่างกันระหว่างวิดีโอจริงและวิดีโอปลอม ผ่านการปรับน้ำหนัก (Weight) ของแบบจำลองซ้ำหลายรอบ (2) Validation Set คิดเป็นร้อยละ 15 ของข้อมูลทั้งหมด ใช้ตรวจสอบประสิทธิภาพของแบบจำลองระหว่างการฝึกในแต่ละ Epoch เพื่อปรับแต่งค่าพารามิเตอร์ เช่น Learning Rate ให้เหมาะสม และใช้เป็นเกณฑ์ในการเลือกน้ำหนักแบบจำลองที่ดีที่สุด (Best Checkpoint) โดยไม่นำข้อมูลชุดนี้ไปใช้ปรับน้ำหนักแบบจำลองโดยตรง (3) Testing Set คิดเป็นร้อยละ 15 ของข้อมูลทั้งหมด เป็นชุดข้อมูลที่แบบจำลองไม่เคยพบทั้งในขั้นตอนการฝึกและการปรับแต่งค่าพารามิเตอร์ ใช้สำหรับประเมินประสิทธิภาพขั้นสุดท้ายของแบบจำลองด้วยตัวชี้วัดต่าง ๆ เพื่อยืนยันว่าแบบจำลองสามารถทำงานได้อย่างแม่นยำกับข้อมูลใหม่ที่ไม่เคยเห็นมาก่อน ทั้งนี้ การแบ่งชุดข้อมูลดำเนินการในระดับวิดีโอ (Video-level Split) ก่อนทำการสกัดเฟรม เพื่อป้องกันไม่ให้เฟรมจากวิดีโอเดียวกันปรากฏทั้งใน Training Set และ Testing Set ซึ่งจะทำให้ผลการประเมินสูงเกินความเป็นจริง (Data Leakage)

3.4.5 การขยายบริบท (Context Expansion)

การขยายบริบท เป็นเทคนิคที่ผู้พัฒนานำเสนอเพื่อเพิ่มประสิทธิภาพในการตรวจจับ Deepfake โดยมีแนวคิดพื้นฐานว่ากระบวนการสร้าง Deepfake ด้วยเทคนิคต่าง ๆ เช่น Face Swap หรือ Face Reenactment มักทิ้งร่องรอยความผิดปกติไว้บริเวณขอบใบหน้าที่เชื่อมต่อกับส่วนอื่นของภาพ เช่น รอยต่อของสีผิว แสงเงาที่ไม่สอดคล้องกัน หรือขอบภาพที่เบลอผิดธรรมชาติ ซึ่งบริเวณดังกล่าวมักถูกตัดทิ้งไปหากใช้เพียงกรอบใบหน้าที่ตรวจจับได้แบบพอดี (Tight Bounding Box) จากขั้นตอน Face Detection เพียงอย่างเดียว ในงานวิจัยนี้จึงทำการขยายกรอบใบหน้าที่ตรวจจับได้จาก RetinaFace ออกไปเพิ่มอีก 35% ในทุกทิศทาง เพื่อให้ภาพที่นำเข้าแบบจำลองครอบคลุมทั้งบริเวณใบหน้าและพื้นที่โดยรอบ (1) อัตราส่วนการขยาย กำหนดไว้ที่เพิ่มอีก 35% จากกรอบใบหน้าเดิม โดยคำนวณความกว้างและความสูงใหม่จากความกว้างและความสูงเดิมคูณด้วย 1.35 ตามลำดับ ค่านี้ถูกเลือกเพื่อให้ครอบคลุมบริเวณเส้นผม ลำคอบางส่วน และพื้นหลังโดยรอบใบหน้า ซึ่งเป็นบริเวณที่

กระบวนการสร้าง Deepfake มักทิ้งร่องรอยความผิดปกติไว้ โดยไม่กว้างจนรวมข้อมูลพื้นหลังที่ไม่เกี่ยวข้องมากเกินไป (2) จุดอ้างอิงการขยาย ใช้จุดศูนย์กลางของกรอบใบหน้าเดิมเป็นจุดอ้างอิงในการคำนวณกรอบใหม่ เพื่อให้ใบหน้ายังคงอยู่กึ่งกลางของภาพหลังการขยาย ไม่เกิดการเลื่อนตำแหน่งที่อาจทำให้แบบจำลองเรียนรู้ผิดจุด (3) การจัดการขอบภาพ ในกรณีที่กรอบภาพที่ขยายแล้วเกินขอบเขตของภาพต้นฉบับ เช่น ใบหน้าอยู่ใกล้ขอบของเฟรม ระบบจะทำการ Padding ด้วยค่าสีดำเพื่อรักษาอัตราส่วนของภาพให้คงที่ ก่อนปรับขนาดเป็น 299 × 299 พิกเซล และนำเข้าสู่แบบจำลอง XceptionNet ในขั้นตอนต่อไป



รูปที่ 3.6 การเปรียบเทียบกรอบใบหน้าก่อนและหลังการขยายบริบท (Context Expansion)

3.4.6 สถาปัตยกรรมโมเดล (Model Architecture)

โครงงานนี้ใช้เทคนิค Transfer Learning โดย Fine-tune บน XceptionNet ที่ Pre-trained ด้วย ImageNet ดังนี้

1) โหลด XceptionNet Base Model (Pre-trained on ImageNet) พร้อม Freeze Layers ในช่วงแรก

2) ใช้ Global Average Pooling ใน Exit Flow เพื่อลดมิติ Feature Map ให้เป็นเวกเตอร์

3) เพิ่ม Dense Layer (128 units, ReLU Activation) + Dropout (0.5)

4) เพิ่ม Output Layer (2 units, Sigmoid Activation) สำหรับจำแนก Real และ Fake

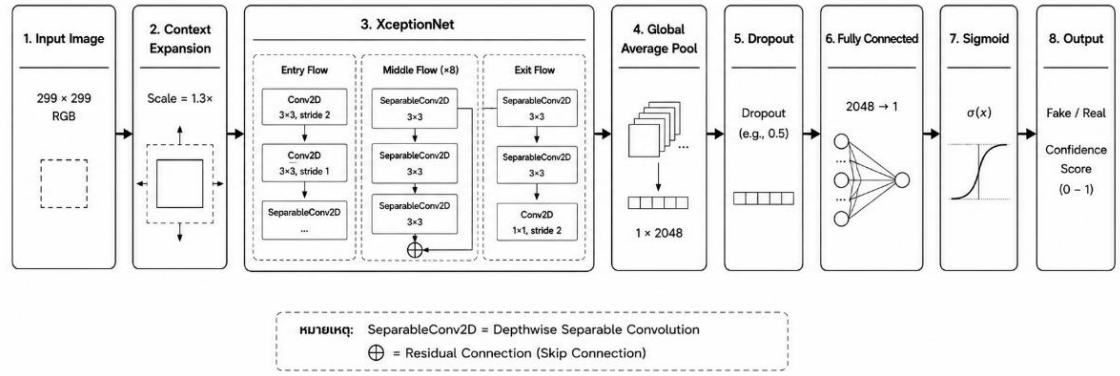
5) ฝึกสอน Classification Head ก่อน จากนั้น Unfreeze Base Model เพื่อ Fine-tune ทั้งหมด

6) บันทึกโมเดลที่ฝึกสอนแล้วเป็นไฟล์ PyTorch Format (.pth) สำหรับ Server-side Inference ผ่าน FastAPI

โมเดล XceptionNet ที่ใช้ในระบบพัฒนาด้วยแนวทาง Transfer Learning โดยนำโมเดลที่ผ่านการฝึกสอนบนชุดข้อมูล ImageNet มาใช้เป็นฐานการเรียนรู้ เพื่อช่วยลดเวลาในการฝึกสอนและเพิ่มประสิทธิภาพในการจำแนกภาพใบหน้าจากชุดข้อมูล FaceForensics++ ที่มีจำนวนจำกัดเมื่อเทียบกับ ImageNet

กระบวนการเริ่มจาก Input Image ซึ่งรับภาพใบหน้าขนาด 299×299 พิกเซลในรูปแบบ RGB ที่ผ่านการขยายบริบทด้วย Context Expansion แล้ว จากนั้นผ่านการ Normalize ก่อนเข้าสู่ Entry Flow เพื่อสกัดคุณลักษณะพื้นฐานของภาพ จากนั้นข้อมูลจะเข้าสู่ส่วน Middle Flow ซึ่งประกอบด้วยบล็อก Depthwise Separable Convolution จำนวน 8 บล็อกต่อเนื่องกัน ทำหน้าที่เรียนรู้ลักษณะเชิงลึกของภาพ เช่น รอยต่อของสีผิว ความไม่ต่อเนื่องของแสงเงา และรายละเอียดเฉพาะบริเวณขอบใบหน้าที่มักเกิดจากกระบวนการสร้าง Deepfake โดยใช้เทคนิค Depthwise Separable Convolution ซึ่งช่วยเพิ่มประสิทธิภาพในการสกัดคุณลักษณะและลดจำนวนพารามิเตอร์ของโมเดล หลังผ่าน Middle Flow แล้ว ข้อมูลจะเข้าสู่ Exit Flow และ Global Average Pooling เพื่อสรุป Feature Map ให้อยู่ในรูปแบบเวกเตอร์พร้อมสำหรับการจำแนกผล จากนั้นเข้าสู่ส่วน Classification Head (Fine-tuned Layers) ซึ่งเป็นชั้นที่ปรับแต่งเพิ่มเติมสำหรับงานวิจัยนี้โดยเฉพาะ ประกอบด้วย Dense Layer, Dropout เพื่อช่วยลดปัญหา Overfitting และเพิ่มเสถียรภาพในการฝึกสอน ก่อนส่งผลลัพธ์ผ่าน Output Layer แบบ Sigmoid เพื่อคำนวณค่า Probability และ Confidence Score ว่าภาพนั้นเป็น Real หรือ Fake ผลลัพธ์สุดท้ายของโมเดลคือค่าความน่าจะเป็นที่ภาพหรือวิดีโอเป็น Fake พร้อมค่าความเชื่อมั่นของแต่ละคลาส ซึ่งช่วยให้ระบบสามารถตรวจสอบสื่อได้อย่างรวดเร็ว ใช้ทรัพยากรต่ำ และตอบสนองต่อผู้ใช้งานได้แบบใกล้เคียงเรียลไทม์ นอกจากนี้โครงสร้างแบบ Fine-tuning ยังเปิดโอกาสให้สามารถปรับปรุงและเพิ่มความแม่นยำของโมเดลในอนาคตได้โดยไม่ต้องสร้างโมเดลใหม่ทั้งหมด โมเดล XceptionNet ถูกเลือกใช้เนื่องจากเป็นสถาปัตยกรรมที่มีประสิทธิภาพสูงในการจำแนกภาพ โดยใช้เทคนิค Depthwise Separable Convolution ที่ช่วยลดจำนวนพารามิเตอร์ของโมเดลได้เมื่อเทียบกับโครงสร้าง CNN แบบดั้งเดิม แต่ยังคงรักษาระดับความแม่นยำในการทำนายไว้ได้อย่างเหมาะสม การใช้ Transfer Learning จากชุดข้อมูล ImageNet ยังช่วยลดระยะเวลาในการฝึกสอนและเพิ่มประสิทธิภาพของการเรียนรู้สำหรับชุดข้อมูล FaceForensics++ ที่มีขนาดจำกัดเมื่อเทียบกับ ImageNet ในกระบวนการฝึกสอน ผู้พัฒนาได้ปรับแต่งเฉพาะชั้น Classification Head เพื่อให้โมเดลสามารถเรียนรู้ลักษณะเฉพาะของร่องรอย Deepfake ในภาพใบหน้าได้ดีขึ้น พร้อมใช้ Dropout เพื่อช่วยลดปัญหา Overfitting และเพิ่มเสถียรภาพของผลลัพธ์ระหว่างการฝึกสอน นอกจากนี้การใช้ Sigmoid Activation ยังช่วยให้สามารถแสดงผลในรูปแบบความน่าจะเป็นของแต่ละคลาสได้อย่างชัดเจน

การบันทึกโมเดลเป็นไฟล์ PyTorch Format (.pth) ยังช่วยให้สามารถโหลดน้ำหนักโมเดลไปใช้งานบนฝั่ง Server (FastAPI Service) ได้โดยตรง โดยไม่ต้องแปลงรูปแบบเพิ่มเติม ทำให้ระบบสามารถประมวลผลภาพและวิดีโอที่ผู้ใช้อัปโหลดได้อย่างรวดเร็วผ่านการเรียกใช้งานแบบ API



รูปที่ 3.7 สถาปัตยกรรมโมเดล XceptionNet ที่ปรับแต่งสำหรับงานตรวจจับ Deepfake

3.4.7 การกำหนดค่าการฝึกสอน (Training Configuration)

การฝึกสอนแบบจำลอง XceptionNet ดำเนินการบนแพลตฟอร์ม Kaggle โดยใช้ GPU Tesla P100 สำหรับการคำนวณ มีขั้นตอนการกำหนดค่าดังนี้

1) โหลดน้ำหนักแบบจำลอง XceptionNet ที่ผ่านการ Pre-trained จากไลบรารี timm พร้อม Freeze น้ำหนักของ Backbone ในช่วงแรก

2) แทนที่ชั้น Classification Head เดิมด้วยชั้น Dense (128 units) + Dropout (0.5) + Output Layer (Sigmoid) ที่ปรับแต่งใหม่

3) ฝึกสอนเฉพาะ Classification Head เป็น Warm-up ก่อน ด้วย Learning Rate = 1e-4 เป็นจำนวน 5 Epoch

4) Unfreeze ชั้นท้าย ๆ ของ Backbone จากนั้นทำ Fine-tuning ทั้งแบบจำลองต่ออีก 15 Epoch

5) ใช้ ReduceLROnPlateau Scheduler ลด Learning Rate อัตโนมัติเมื่อ Validation Loss ไม่ดีขึ้นติดต่อกัน 3 Epoch

6) บันทึกน้ำหนักแบบจำลองที่ดีที่สุด (Best Checkpoint) ตาม Validation Loss ต่ำสุดตลอดการฝึกสอน

ตารางที่ 3.11 พารามิเตอร์การกำหนดค่าการฝึกสอนแบบจำลอง XceptionNet

พารามิเตอร์	ค่าที่กำหนด	คำอธิบาย
Optimizer	Adam	ปรับ Learning Rate ของแต่ละพารามิเตอร์อัตโนมัติ เรียนรู้เร็วและเสถียรกว่า SGD
Learning Rate	1e-4	ตั้งค่าต่ำเพราะเป็นการ Fine-tune โมเดลที่ Pre-trained มาแล้ว ป้องกัน Overfitting
LR Scheduler	ReduceLROnPlateau	ลด Learning Rate อัตโนมัติเมื่อ Validation Loss ไม่ดีขึ้นติดต่อกัน
Loss Function	Binary Cross-Entropy	เหมาะกับการจำแนก 2 คลาส (Real/Fake)
Batch Size	32	สมดุลระหว่างความเร็วในการฝึกกับหน่วยความจำ GPU ที่มีจำกัด
จำนวน Epoch	20 (Warm-up 5 + Fine-tune 15)	เพียงพอให้แบบจำลอง Convergence โดยไม่ Overfitting
Dropout Rate	0.5	ลดการพึ่งพาเซลล์ใดเซลล์หนึ่งในชั้น Dense ป้องกัน Overfitting

ในกระบวนการฝึกสอน ผู้พัฒนาแบ่งการฝึกออกเป็น 2 ช่วงเพื่อรักษาความรู้เดิมของแบบจำลองที่ผ่านการ Pre-trained บน ImageNet ไว้ให้มากที่สุด โดยช่วงแรกฝึกสอนเฉพาะ Classification Head ที่เพิ่มขึ้นใหม่เท่านั้น เนื่องจากชั้นนี้เริ่มต้นด้วยน้ำหนักแบบสุ่ม (Random Initialization) จึงจำเป็นต้องเรียนรู้ก่อนเป็นลำดับแรกโดยไม่รบกวนน้ำหนักของ Backbone ที่ผ่านการฝึกมาแล้ว หลังจากนั้นในช่วงที่สองจึงทำการ Unfreeze ชั้นท้าย ๆ ของ Backbone เพื่อ Fine-tuning ร่วมกับ Classification Head ทั้งหมด โดยใช้ Learning Rate เดียวกันแต่ปรับลดอัตโนมัติผ่าน ReduceLROnPlateau เมื่อ Validation Loss เริ่มคงที่ การใช้ Binary

Cross-Entropy เป็น Loss Function เหมาะสมกับลักษณะของปัญหาที่เป็นการจำแนกแบบ 2 คลาส (Real และ Fake) โดยแบบจำลองจะเรียนรู้ให้ค่าความน่าจะเป็นที่ทำนายออกมาใกล้เคียงกับป้ายกำกับจริงมากที่สุด ส่วนการกำหนด Batch Size ที่ 32 ภาพเป็นค่าที่สมดุลระหว่างความเร็วในการฝึกสอนกับข้อจำกัดของหน่วยความจำ GPU บนแพลตฟอร์ม Kaggle ทั้งนี้ตลอดกระบวนการฝึกสอน ระบบจะบันทึกน้ำหนักของแบบจำลองที่ให้ค่า Validation Loss ต่ำที่สุดไว้เป็น Best Checkpoint เพื่อนำไปใช้ในการประเมินผลและใช้งานจริงในขั้นตอนถัดไป

3.4.8 ตัวชี้วัดการประเมินผล (Evaluation Metrics)

การประเมินประสิทธิภาพของแบบจำลองใช้ตัวชี้วัดมาตรฐานสำหรับงานจำแนกประเภทแบบ 2 คลาส (Binary Classification) เพื่อวัดความสามารถของแบบจำลองในการแยกแยะวิดีโอจริงและวิดีโอปลอมจากหลายมุมมอง โดยแต่ละตัวชี้วัดคำนวณจากผลการทำนายบน Testing Set ที่แบบจำลองไม่เคยพบมาก่อน ร่วมกับค่า True Positive (TP), True Negative (TN), False Positive (FP) และ False Negative (FN) ที่ได้จาก Confusion Matrix

ตารางที่ 3.12 ตัวชี้วัดที่ใช้ในการประเมินประสิทธิภาพแบบจำลอง

ลำดับ	ตัวชี้วัด	สูตรการคำนวณ	คำอธิบาย
1	Accuracy	(TP + TN) / (TP + TN + FP + FN)	สัดส่วนการทำนายที่ถูกต้องทั้งหมดต่อข้อมูลทั้งหมด
2	Precision	TP / (TP + FP)	ความแม่นยำเมื่อแบบจำลองทำนายว่าเป็น Fake
3	Recall	TP / (TP + FN)	ความสามารถในการตรวจจับวิดีโอ Fake ที่มีอยู่จริงได้ครบถ้วน
4	F1-Score	2×(Precision × Recall) / (Precision + Recall)	ค่าเฉลี่ยฮาร์โมนิกระหว่าง Precision และ Recall
5	AUC-ROC	พื้นที่ใต้กราฟ ROC Curve	ความสามารถในการจำแนกโดยไม่ขึ้นกับค่า Threshold

(1) Accuracy คือสัดส่วนของการทำนายที่ถูกต้องทั้งหมดต่อจำนวนข้อมูลทั้งหมด ใช้วัดความแม่นยำโดยรวมของแบบจำลอง แต่มีข้อจำกัดเมื่อชุดข้อมูลมีความไม่สมดุลระหว่างคลาส (2) Precision คืออัตราส่วนของผลบวกที่ทำนายถูกต้องต่อผลบวกทั้งหมดที่แบบจำลองทำนาย ใช้วัดว่าเมื่อแบบจำลองทำนายว่าเป็น Fake มีความน่าเชื่อถือเพียงใด ค่านี้สำคัญในบริบทของงานตรวจสอบข้อเท็จจริง เพราะการแจ้งเตือนวิดีโอ Real ว่าเป็น Fake (False Positive) อาจสร้างความเข้าใจผิดต่อผู้ใช้งานได้ (3) Recall คืออัตราส่วนของผลบวกที่ทำนายถูกต้องต่อผลบวกทั้งหมดที่มีอยู่จริงในข้อมูล ใช้วัดว่าแบบจำลองสามารถตรวจจับวิดีโอ Fake ที่มีอยู่จริงได้ครบถ้วนเพียงใด ซึ่งสำคัญเช่นกัน เพราะการปล่อยให้วิดีโอ Fake ผ่านไปโดยไม่ถูกตรวจพบ (False Negative) อาจก่อให้เกิดผลกระทบร้ายแรงต่อความน่าเชื่อถือของข่าว (4) F1-Score คือค่าเฉลี่ยฮาร์โมนิกระหว่าง Precision และ Recall ใช้เป็นตัวชี้วัดเดียวที่สะท้อนความสมดุลระหว่างสองค่าดังกล่าว เหมาะสำหรับกรณีที่ต้องพิจารณาทั้ง Precision และ Recall ควบคู่กัน และ (5) AUC-ROC คือพื้นที่ใต้กราฟ ROC ที่แสดงความสัมพันธ์ระหว่าง True Positive

Rate และ False Positive Rate ที่ค่า Threshold ต่าง ๆ ใช้วัดความสามารถโดยรวมของแบบจำลองในการแยกข้อมูลจริงและข้อมูลปลอม โดยค่า AUC ที่เข้าใกล้ 1.0 บ่งชี้ว่าแบบจำลองมีความสามารถในการจำแนกที่ดี

นอกจากตัวชี้วัดเชิงตัวเลขทั้ง 5 รายการแล้ว งานวิจัยนี้ยังใช้ Confusion Matrix เพื่อวิเคราะห์ความผิดพลาดของแบบจำลองในระดับคลาส เพื่อระบุว่าแบบจำลองมีแนวโน้มทำนายผิดพลาดไปในทิศทางใดระหว่าง False Positive และ False Negative ซึ่งมีนัยสำคัญต่างกันในบริบทของงานตรวจสอบข้อเท็จจริงในสื่อมวลชนดังที่กล่าวไว้ข้างต้น

3.5 การออกแบบฐานข้อมูล (Database Design)

ระบบเว็บแอปพลิเคชันตรวจจับ Deepfake ใช้ PostgreSQL เป็นระบบจัดการฐานข้อมูลเชิงสัมพันธ์ (Relational Database Management System) โดยเข้าถึงและจัดการข้อมูลผ่าน Prisma ORM เพื่อให้การเขียนและอ่านข้อมูลมีความปลอดภัยด้วยการตรวจสอบชนิดข้อมูล (Type Safety) ฐานข้อมูลถูกออกแบบให้รองรับการจัดเก็บข้อมูลสำคัญ 3 ส่วนหลัก ได้แก่ ข้อมูลบัญชีผู้ใช้งาน ข้อมูลงานตรวจสอบ และข้อมูลผลการวิเคราะห์รายเฟรม

ตารางที่ 3.13 ตาราง User (ข้อมูลบัญชีผู้ใช้งาน)

ลำดับ	ชื่อฟิลด์	ชนิดข้อมูล	Null	ค่าเริ่มต้น	คำอธิบาย
1	uId	UUID (PK)	No	Auto-generated	รหัสผู้ใช้งาน (Primary Key)
2	username	VARCHAR(100) UNIQUE	No	-	ชื่อผู้ใช้ที่แสดงในระบบ ต้องไม่ซ้ำกัน
3	email	VARCHAR(255) UNIQUE	No	-	อีเมลที่ใช้สมัครสมาชิก ต้องไม่ซ้ำกัน
4	passwordHash	VARCHAR(255)	No	-	รหัสผ่านที่ผ่านการเข้ารหัสด้วย bcrypt
5	role	ENUM('member','admin')	No	'member'	บทบาทของผู้ใช้งาน
6	createdAt	TIMESTAMP	No	now()	วันและเวลาที่สมัครสมาชิก
7	updatedAt	TIMESTAMP	No	now()	วันและเวลาที่อัปเดตข้อมูลล่าสุด

ตารางที่ 3.14 ตาราง DetectionJob (ข้อมูลงานตรวจสอบ)

ลำดับ	ชื่อฟิลด์	ชนิดข้อมูล	Null	ค่าเริ่มต้น	คำอธิบาย
1	id	UUID (PK)	No	Auto-generated	รหัสงานตรวจสอบ
2	uId	UUID (FK)	Yes	NULL	รหัสผู้ใช้ (NULL สำหรับ Guest User)
3	fileType	ENUM('image','video')	No	-	ประเภทไฟล์ที่ตรวจสอบ
4	originalFileName	VARCHAR(500)	No	-	ชื่อไฟล์เดิมของผู้ใช้

ตารางที่ 3.14(ต่อ) ตาราง DetectionJob (ข้อมูลงานตรวจสอบ)

ลำดับ	ชื่อฟิลด์	ชนิดข้อมูล	Null	ค่าเริ่มต้น	คำอธิบาย
5	storagePath	VARCHAR(500)	No	-	ที่อยู่ไฟล์ใน Cloudflare R2
6	status	ENUM('pending', 'processing','done','failed')	No	'pending'	สถานะของงานตรวจสอบ
7	result	ENUM('real','fake')	Yes	NULL	ผลการตรวจสอบ
8	confidenceScore	FLOAT	Yes	NULL	ค่าความเชื่อมั่นของผล (0.0 – 1.0)
9	processingTimeMs	INTEGER	Yes	NULL	เวลาที่ใช้ในการประมวลผล
10	createdAt	TIMESTAMP	No	now()	วันและเวลาที่สร้างงานตรวจสอบ

ตารางที่ 3.15 ตาราง FrameResult (ข้อมูลผลรายเฟรม)

ลำดับ	ชื่อฟิลด์	ชนิดข้อมูล	Null	ค่าเริ่มต้น	คำอธิบาย
1	id	UUID (PK)	No	Auto-generated	รหัสผลรายเฟรม
2	jobId	UUID (FK)	No	-	รหัสงานตรวจสอบ
3	frameIndex	INTEGER	No	-	ลำดับเฟรมในวิดีโอ เริ่มจาก 0
4	timestampSec	FLOAT	No	-	เวลาของเฟรมในวิดีโอ
5	fakeScore	FLOAT	No	-	ค่าความน่าจะเป็นที่เป็น Fake (0.0 – 1.0)
6	faceDetected	BOOLEAN	No	false	สถานะการพบใบหน้าในเฟรมนี้

ความสัมพันธ์ระหว่างตารางในฐานข้อมูลเป็นแบบ One-to-Many กล่าวคือ ผู้ใช้งาน 1 คน (User) สามารถมีงานตรวจสอบได้หลายงาน (DetectionJob) และงานตรวจสอบ 1 งานสามารถมีผลรายเฟรมได้หลายรายการ (FrameResult) สำหรับกรณีที่ตรวจสอบไฟล์วิดีโอ โดยกำหนด Cascade Delete เพื่อให้ข้อมูลที่เกี่ยวข้องถูกลบโดยอัตโนมัติเมื่อมีการลบข้อมูลในตารางหลัก

ตารางที่ 3.16 ตาราง SystemLog (บันทึกการใช้งานระบบ)

ลำดับ	ชื่อฟิลด์	ชนิดข้อมูล	Null	ค่าเริ่มต้น	คำอธิบาย
1	logId	UUID (PK)	No	Auto-generated	รหัสบันทึกกิจกรรม (Primary Key)
2	uId	UUID (FK)	Yes	NULL	รหัสผู้ใช้ที่ทำกิจกรรม
3	action	VARCHAR(100)	No	-	ประเภทกิจกรรมที่เกิดขึ้น

ตารางที่ 3.16(ต่อ) ตาราง SystemLog (บันทึกการใช้งานระบบ)

ลำดับ	ชื่อฟิลด์	ชนิดข้อมูล	Null	ค่าเริ่มต้น	คำอธิบาย
4	description	VARCHAR(500)	Yes	NULL	รายละเอียดเพิ่มเติมของกิจกรรมนั้น
5	ipAddress	VARCHAR(45)	Yes	NULL	IP Address ของผู้ใช้งานขณะทำกิจกรรม
6	userAgent	VARCHAR(255)	Yes	NULL	ข้อมูล Browser และอุปกรณ์ที่ใช้งานขณะทำกิจกรรม
7	createdAt	TIMESTAMP	No	now()	วันและเวลาที่เกิดกิจกรรม

ตารางที่ 3.17 ตารางประเภทกิจกรรม (Action Types) ที่บันทึกในระบบ

ลำดับ	Action	เมื่อเกิดขึ้น
1	LOGIN	ผู้ใช้เข้าสู่ระบบสำเร็จ
2	LOGIN_FAILED	ผู้ใช้เข้าสู่ระบบไม่สำเร็จ (รหัสผ่านผิด)
3	LOGOUT	ผู้ใช้ออกจากระบบ
4	UPLOAD_FILE	ผู้ใช้อัปโหลดไฟล์เพื่อตรวจสอบ
5	VIEW_HISTORY	ผู้ใช้เข้าดูหน้าประวัติการตรวจสอบ
6	VERIFY_RESULT	Admin ยืนยันผลการตรวจสอบ
7	ADMIN_LOGIN	Admin เข้าสู่ระบบ Dashboard

3.5.1 พจนานุกรมข้อมูล (Data Dictionary)

พจนานุกรมข้อมูลจัดทำขึ้นเพื่อกำหนดรายละเอียดของแต่ละฟิลด์ในฐานข้อมูลให้ชัดเจนยิ่งขึ้น ครอบคลุมเงื่อนไข ค่าเริ่มต้น และข้อจำกัดของข้อมูล เพื่อใช้เป็นมาตรฐานอ้างอิงในการพัฒนาและบำรุงรักษาระบบ

ตารางที่ 3.18 พจนานุกรมข้อมูล (Data Dictionary) ของระบบ

ลำดับ	ตาราง.ฟิลด์	ข้อจำกัด	ค่าเริ่มต้น	คำอธิบายเพิ่มเติม
1	User.email	NOT NULL, UNIQUE	-	ต้องผ่านการตรวจสอบรูปแบบอีเมลก่อนบันทึก
2	User.passwordHash	NOT NULL	-	ความยาวรหัสผ่านต้นฉบับขั้นต่ำ 8 ตัวอักษรก่อนเข้ารหัส
3	User.role	NOT NULL	'member'	กำหนดเป็น admin ได้เฉพาะผ่านฐานข้อมูลโดยตรงเท่านั้น
4	DetectionJob.fileType	NOT NULL	-	ตรวจสอบจาก MIME Type ของไฟล์ที่รับเข้า

ตารางที่ 3.18(ต่อ) พจนานุกรมข้อมูล (Data Dictionary) ของระบบ

ลำดับ	ตาราง.ฟิลด์	ข้อจำกัด	ค่าเริ่มต้น	คำอธิบายเพิ่มเติม
5	DetectionJob.status	NOT NULL	'pending'	เปลี่ยนสถานะตามลำดับ pending, processing, done/failed
6	DetectionJob.confidenceScore	CHECK (0 ≤ value ≤ 1)	NULL	มีค่าเมื่อ status = done เท่านั้น
7	FrameResult.frameIndex	NOT NULL, CHECK (≥ 0)	-	เรียงลำดับตามเวลาที่ปรากฏในวิดีโอ
8	FrameResult.jobId	NOT NULL, FK ON DELETE CASCADE	-	ลบอัตโนมัติเมื่อ DetectionJob ต้นทางถูกลบ

ตารางที่ 3.19 สรุปตารางและความสัมพันธ์ในฐานข้อมูล

ลำดับ	ชื่อตาราง	จำนวน Fields	ความสัมพันธ์หลัก
1	User	6	เชื่อมกับ DetectionJob และ SystemLog ผ่าน uId (1:N)
2	DetectionJob	10	เชื่อมกับ User ผ่าน uId (1:N) และเชื่อมกับ FrameResult ผ่าน jobId (1:N)
3	FrameResult	6	เชื่อมกับ DetectionJob ผ่าน jobId (N:1)
4	SystemLog	7	เชื่อมกับ User ผ่าน uId (1:N)

3.6 การออกแบบซอฟต์แวร์ (Software Module Design)

การออกแบบซอฟต์แวร์ของระบบยึดหลัก Modular Design โดยแบ่งโมดูลตามหน้าที่การทำงานที่ชัดเจน แต่ละโมดูลรับผิดชอบงานเฉพาะด้านและสื่อสารกันผ่าน Interface ที่กำหนดไว้ล่วงหน้า ทำให้สามารถพัฒนาและทดสอบแต่ละส่วนได้อย่างอิสระ แบ่งออกเป็น 2 กลุ่มหลัก ได้แก่ โมดูลฝั่ง Frontend (Next.js) และโมดูลฝั่ง Backend (FastAPI)

ตารางที่ 3.20 โมดูลซอฟต์แวร์ฝั่ง Frontend (Next.js)

ลำดับ	โมดูล	หน้าที่การทำงาน
1	Authentication Module	จัดการสมัครสมาชิก เข้าสู่ระบบ และสิทธิ์การเข้าถึงด้วย JWT
2	Upload Module	รับไฟล์ภาพและวิดีโอ พร้อมตรวจสอบประเภทและขนาดก่อนส่ง
3	Detection Result Module	แสดงผล Real/Fake, Confidence Score และ Timeline Chart
4	History Module	แสดงและกรองประวัติการตรวจสอบของสมาชิก
5	Admin Dashboard Module	แสดงสถิติภาพรวมและประวัติการตรวจสอบทั้งระบบ

ตารางที่ 3.21 โมดูลซอฟต์แวร์ฝั่ง Backend (FastAPI)

ลำดับ	โมดูล	หน้าที่การทำงาน
1	Auth Module	ตรวจสอบและออก JWT Token พร้อมแยกสิทธิ์ตาม Role
2	File Handler Module	ตรวจสอบไฟล์และอัปโหลดไปยัง Cloudflare R2
3	Deepfake Detection Module	ประสาน Pipeline ตรวจจับตั้งแต่แยกเฟรมถึง Inference
4	XceptionNet Inference Module	โหลดโมเดลและคำนวณ Fake Score ของภาพ
5	Log Module	บันทึกกิจกรรมผู้ใช้งานลงตาราง SystemLog
6	Database Module (Prisma)	จัดการ CRUD ผ่าน Prisma ORM ทุกตาราง

3.7 การออกแบบเชิงพฤติกรรม (Behavioral Design)

การออกแบบเชิงพฤติกรรมอธิบายลำดับการทำงานของผู้ใช้และระบบในกรณีการใช้งานหลัก โดยใช้ทั้ง Activity Diagram เพื่อแสดงลำดับขั้นตอนการทำงานในเชิงกระบวนการ และ Sequence Diagram เพื่อแสดงการสื่อสารระหว่างส่วนประกอบต่าง ๆ ของระบบ

3.7.1 Activity Diagram: กระบวนการตรวจสอบ Deepfake

Activity Diagram แสดงลำดับขั้นตอนการทำงานของผู้ใช้งานและระบบตั้งแต่เริ่มต้นจนสิ้นสุดกระบวนการตรวจสอบ Deepfake โดยเริ่มจากผู้ใช้งาน (Guest หรือ Member) เปิดเว็บแอปพลิเคชันและเลือกไฟล์ภาพหรือวิดีโอที่ต้องการตรวจสอบ ระบบจะตรวจสอบประเภทและขนาดไฟล์ก่อนเป็นอันดับแรก หากไฟล์ไม่ผ่านเงื่อนไข (เช่น เกิน 100 MB หรือประเภทไฟล์ไม่รองรับ) ระบบจะแสดงข้อความแจ้งเตือนและย้อนกลับไปให้ผู้ใช้เลือกไฟล์ใหม่ หากไฟล์ผ่านเงื่อนไข ระบบจะอัปโหลดไฟล์ไปยัง Cloudflare R2 และส่งต่อไปยังกระบวนการตรวจจับ ในกระบวนการตรวจจับ ระบบจะแยกเป็น 2 เส้นทางคู่ขนานตามประเภทไฟล์ หากเป็นวิดีโอจะทำ Frame Extraction ก่อน ส่วนภาพนิ่งจะข้ามขั้นตอนนี้ไปยัง Face Detection โดยตรง จากนั้นทั้งสองเส้นทางจะเข้าสู่ขั้นตอน Context Expansion และ XceptionNet Inference เหมือนกัน เมื่อได้ผลลัพธ์แล้ว ระบบจะแสดงผล Real/Fake พร้อม Confidence Score ให้ผู้ใช้งาน และหาก Member เข้าสู่ระบบอยู่ ระบบจะบันทึกประวัติลงตาราง DetectionJob โดยอัตโนมัติเป็นขั้นตอนสุดท้ายก่อนจบกระบวนการ

รูปที่ 3.8 Activity Diagram กระบวนการตรวจสอบ Deepfake

3.7.2 Sequence Diagram: กระบวนการตรวจสอบ Deepfake

กระบวนการตรวจสอบ Deepfake เริ่มต้นเมื่อผู้ใช้งาน (Guest หรือ Member) เปิดเว็บแอปพลิเคชันและอัปโหลดไฟล์ภาพหรือวิดีโอ โดยมีลำดับการสื่อสารระหว่างส่วนประกอบของระบบ ดังนี้ (1) ผู้ใช้เลือกไฟล์และกดปุ่ม "ตรวจสอบ" บน Browser Frontend จะตรวจสอบประเภทและขนาดไฟล์เบื้องต้นก่อน (2) Frontend ส่ง HTTP POST พร้อม FormData ไปยัง Backend API (3) Backend ตรวจสอบ JWT Token ในกรณี Member อัปโหลดไฟล์ไปยัง Cloudflare R2 และสร้างรายการ DetectionJob ใน PostgreSQL (4) Backend เรียกใช้ Deepfake Detection Module เพื่อเริ่มกระบวนการวิเคราะห์ (5) Detection Module ดำเนินการตามลำดับ 4 ขั้นตอน ได้แก่ Frame Extraction, Face Detection, Context Expansion และ XceptionNet Inference ตามลำดับ (6) ผลการวิเคราะห์ถูกส่งกลับมายัง Backend เพื่ออัปเดตสถานะในตาราง DetectionJob

และบันทึกข้อมูลลงตาราง FrameResult (7) Backend เรียกใช้ Log Module เพื่อบันทึกกิจกรรม UPLOAD_FILE ลงตาราง SystemLog (8) Backend ส่งผลการตรวจสอบกลับไปยัง Frontend และ (9) Frontend แสดงผล Real หรือ Fake พร้อม Confidence Score และ Timeline Chart แก่ผู้ใช้งาน

รูปที่ 3.9 Sequence Diagram กระบวนการตรวจสอบ Deepfake

3.7.3 Sequence Diagram: กระบวนการสมัครสมาชิกและเข้าสู่ระบบ

กระบวนการสมัครสมาชิกและเข้าสู่ระบบแสดงลำดับการสื่อสารระหว่าง Frontend, Backend และฐานข้อมูล PostgreSQL โดยแบ่งออกเป็น 2 กรณี ดังนี้ กรณีสมัครสมาชิก เริ่มจากผู้ใช้กรอกอีเมลและรหัสผ่านบน Frontend จากนั้น Frontend ส่งข้อมูลไปยัง Backend เพื่อตรวจสอบว่าอีเมลดังกล่าวยังไม่มีอยู่ในระบบ หากยังไม่มี Backend จะเข้ารหัสรหัสผ่านด้วย bcrypt บันทึกข้อมูลลงในตาราง User และส่ง JWT Token กลับไปยัง Frontend เพื่อให้ผู้ใช้เข้าสู่ระบบโดยอัตโนมัติหลังสมัครสมาชิกสำเร็จ

กรณีเข้าสู่ระบบ Backend ตรวจสอบอีเมลในฐานข้อมูลและเปรียบเทียบรหัสผ่านที่ผู้ใช้กรอกกับ passwordHash ที่เข้ารหัสไว้ ด้วยฟังก์ชัน bcrypt.compare หากข้อมูลถูกต้อง Backend จะออก JWT Token ที่มีอายุ 24 ชั่วโมงกลับไปยัง Frontend ทั้งนี้ไม่ว่าการเข้าสู่ระบบจะสำเร็จหรือไม่สำเร็จก็ตาม Backend จะเรียกใช้ Log Module เพื่อบันทึกกิจกรรม LOGIN หรือ LOGIN_FAILED ลงตาราง SystemLog ทุกครั้ง เพื่อใช้เป็นหลักฐานตรวจสอบย้อนหลังกรณีมีความพยายามเข้าสู่ระบบที่ผิดปกติ

รูปที่ 3.10 Sequence Diagram กระบวนการสมัครสมาชิกและเข้าสู่ระบบ

3.8 การออกแบบการจัดเก็บข้อมูลและไฟล์ (Data and File Storage Design)

ระบบแยกการจัดเก็บข้อมูลออกเป็น 2 ส่วน ได้แก่ ข้อมูลเชิงโครงสร้างที่จัดเก็บในฐานข้อมูล PostgreSQL และไฟล์สื่อที่จัดเก็บใน Cloudflare R2 Object Storage เพื่อให้ระบบรองรับไฟล์วิดีโอขนาดใหญ่ได้อย่างมีประสิทธิภาพ ลดภาระของฐานข้อมูล และเพิ่มความเร็วในการเข้าถึงไฟล์

ตารางที่ 3.22 โครงสร้างการจัดเก็บไฟล์ใน Cloudflare R2

ลำดับ	Path	หน้าที่การทำงาน	นโยบายการจัดเก็บ
1	uploads/images/{uld}/{uuid}.{ext}	ภาพนิ่งที่อัปโหลดโดย Member	เก็บ 30 วัน ลบอัตโนมัติผ่าน R2 Lifecycle
2	uploads/videos/{uld}/{uuid}.{ext}	วิดีโอที่อัปโหลดโดย Member	เก็บ 30 วัน ลบอัตโนมัติผ่าน R2 Lifecycle
3	uploads/guest/{uuid}.{ext}	ไฟล์ของ Guest User	เก็บ 24 ชั่วโมง ลบอัตโนมัติ
4	models/xceptionnet_ff_c23.pth	น้ำหนักแบบจำลอง XceptionNet	เก็บถาวร อัปเดตเมื่อฝึกสอนโมเดลใหม่

เพื่อความปลอดภัยของข้อมูล ไฟล์ที่อัปโหลดทุกไฟล์จะถูกตั้งชื่อใหม่ด้วย UUID แบบสุ่มแทนชื่อไฟล์เดิมของผู้ใช้ เพื่อป้องกันการเดาชื่อไฟล์ นอกจากนี้ไฟล์ทั้งหมดถูกจัดเก็บแบบ Private Access โดยการ

เข้าถึงไฟล์ต้องผ่าน Pre-signed URL ที่มีอายุจำกัด ทำให้ไม่สามารถเข้าถึงไฟล์โดยตรงจาก URL ปกติได้ ทั้งนี้ Path การจัดเก็บไฟล์ของ Member ใช้ uId เป็นตัวอ้างอิงกลุ่มไฟล์ ให้สอดคล้องกับชื่อฟิลด์ Primary Key ที่กำหนดไว้ในตาราง User

3.9 การออกแบบส่วนติดต่อผู้ใช้ (User Interface Design)

การออกแบบส่วนติดต่อผู้ใช้ (UI) ยึดหลักความเรียบง่าย ใช้งานง่าย และแสดงผลได้ชัดเจน โดยออกแบบด้วย Figma ก่อนนำมาพัฒนาด้วย Tailwind CSS บน Next.js ระบบสีหลักกำหนดให้สีเขียวแสดงผล Real และสีแดงแสดงผล Fake เพื่อให้ผู้ใช้รับรู้ผลลัพธ์ได้ทันทีโดยไม่ต้องอ่านข้อความ ระบบรองรับการแสดงผลบนคอมพิวเตอร์และอุปกรณ์เคลื่อนที่ (Responsive Design) ครอบคลุมหน้าหลักดังนี้

3.11.1 หน้าแรก (Landing Page)

หน้าแรกของระบบ Lanner ออกแบบตามหลักการ Above-the-fold Design ให้เนื้อหาสำคัญที่สุดปรากฏในส่วนที่มองเห็นได้ทันทีโดยไม่ต้องเลื่อนหน้าจอ เพื่อสื่อสารคุณค่าของระบบ ให้ผู้ใช้เข้าใจได้ในไม่กี่วินาทีแรก ซึ่งสำคัญมากสำหรับกลุ่มเป้าหมายอย่างผู้สื่อข่าวที่มักมีเวลาจำกัด ส่วน Hero Section จัดวางตามแนวคิด Z-pattern Layout เริ่มจากป้าย Badge ระบุเทคโนโลยีหลัก "ขับเคลื่อนด้วย XceptionNet + Context Expansion" เพื่อสร้างความน่าเชื่อถือเชิงเทคนิคตั้งแต่แรกเห็น ตามด้วยหัวข้อหลัก "ตรวจสอบวิดีโอปลอม รู้ทัน Deepfake อย่างแม่นยำ" ที่เน้นคำว่า "Deepfake" ด้วยสีเขียวประจำแบรนด์ และคำอธิบายสั้นที่สื่อสารประโยชน์ของระบบแบบ Benefit-driven Copywriting มากกว่าอธิบายฟีเจอร์เชิงเทคนิค

ปุ่มกระทำการออกแบบเป็น 2 ระดับตามหลัก Progressive Disclosure ได้แก่ปุ่มหลัก "เริ่มตรวจจับ" และปุ่มรอง "ดูเพิ่มเติม" เพื่อนำสายตาผู้ใช้ไปยังกระทำการที่ต้องการโดยไม่บังคับ พร้อมป้ายประเภทไฟล์ที่รองรับด้านล่างเพื่อลดความไม่แน่ใจก่อนใช้งาน ด้านขวาแสดงตัวอย่างผลการตรวจสอบจำลอง ตามหลัก Show, Don't Tell ให้ผู้ใช้เห็นลักษณะผลลัพธ์จริงเป็นรูปธรรมก่อนเริ่มใช้งาน ถัดมาเป็นแถบสถิติสรุป 4 รายการ (ความแม่นยำ 97%, ยอดตรวจสอบสะสม 60K+, เทคนิค Deepfake ที่รองรับ 5 แบบ, เวลาประมวลผล ต่ำกว่า 300 มิลลิวินาที) นำหลัก Social Proof มาสร้างความน่าเชื่อถือด้วยตัวเลขที่ตรวจสอบได้ ส่วน Navigation Bar ด้านบนออกแบบให้คงที่ เพื่อให้เข้าถึงปุ่มเข้าสู่ระบบและ "เริ่มต้นใช้งาน" ได้จากทุกตำแหน่งขณะเลื่อนอ่านหน้าเว็บ

รูปที่ 3.11 หน้าแรก (Landing Page) ของระบบ Lanner

3.11.2 หน้าวิธีการทำงาน (How It Works)

หน้านี้ออกแบบเพื่ออธิบายกระบวนการทำงานของระบบให้ผู้ใช้ใหม่เข้าใจได้ง่าย โดยใช้รูปแบบ Numbered Steps 3 ขั้นตอน ได้แก่ (01) อัปโหลดไฟล์ ลากวางหรือเลือกไฟล์รูปภาพหรือวิดีโอ รองรับขนาดสูงสุด 100 MB (02) วิเคราะห์ด้วย AI แบบจำลอง XceptionNet ตรวจจับความผิดปกติบนใบหน้าด้วยเทคนิค Context Expansion เพื่อจับสัญญาณได้มากกว่าบริเวณใบหน้าเพียงอย่างเดียว และ (03) รับผลลัพธ์ แสดงคะแนนความเชื่อมั่นพร้อมรายละเอียดของสัญญาณการปลอมแปลงที่ตรวจพบ การใช้ตัวเลขกำกับแต่ละขั้นตอนช่วยให้ผู้ใช้ประเมินได้ทันทีว่าการใช้งานระบบมีความซับซ้อนเพียงใด ซึ่งเป็นปัจจัยสำคัญที่ส่งผลต่อการตัดสินใจทดลองใช้งานจริง ถัดลงมาแสดงจุดเด่นของระบบในรูปแบบ Card Grid จำนวน 6 การ์ด แบ่งเป็นหมวดหมู่กำกับด้วย Tag ได้แก่ Context Expansion การวิเคราะห์เฟรมวิดีโอ การตรวจจับใบหน้าด้วย RetinaFace พร้อมขยาย

บริบทเพิ่มอีก 35% ก่อนเข้าสู่ขั้นตอนจำแนกผล (กระบวนการ) คะแนนความเชื่อมั่น (ผลลัพธ์) การบันทึกประวัติการตรวจจับ (บัญชีผู้ใช้) และแดชบอร์ดสำหรับผู้ดูแลระบบ (ผู้ดูแลระบบ) การจัดกลุ่มด้วย Tag เช่นนี้ช่วยให้ผู้ใช้แยกแยะได้ว่าแต่ละจุดเด่นอยู่ในมิติใดของระบบ (เทคโนโลยี, กระบวนการ, หรือผลลัพธ์) แทนที่จะแสดงเป็นรายการเรียบ ๆ ที่ไม่มีการจัดหมวดหมู่

รูปที่ 3.12 หน้าวิธีการทำงาน (How It Works) ของระบบ Lanner

3.11.3 หน้าเข้าสู่ระบบและสมัครสมาชิก (Login / Register)

ส่วนติดต่อผู้ใช้ในหัวข้อนี้ประกอบด้วยหน้าเข้าสู่ระบบ และสมัครสมาชิก ออกแบบภายใต้แนวคิดเดียวกัน ใช้รูปแบบ Tab-based Navigation รวมทั้งสองฟอร์มไว้ในการ์ดเดียวกัน เพื่อลดความรู้สึกสะดุดของผู้ใช้ จากการเปลี่ยนหน้า และให้ผู้ใช้ที่กดผิดโหมดสลับไปยังโหมดที่ถูกต้องได้ทันที หน้าเข้าสู่ระบบให้กรอกอีเมลและรหัสผ่าน ซึ่งถูกส่งผ่าน HTTPS ไปตรวจสอบกับค่า passwordHash ที่เข้ารหัสด้วย bcrypt หากถูกต้องระบบจะออก JWT Token จัดเก็บใน HttpOnly Cookie เพื่อป้องกัน Cross-Site Scripting พร้อมลิงก์ "ลืมรหัสผ่าน" สำหรับกรณีผู้ใช้จำรหัสผ่านไม่ได้ ส่วนหน้าสมัครสมาชิกให้กรอกชื่อผู้ใช้ อีเมล รหัสผ่าน และยืนยันรหัสผ่าน โดยฟิลด์ยืนยันรหัสผ่านช่วยลดความผิดพลาดจากการพิมพ์ผิดโดยไม่รู้ตัว เมื่อกดสร้างบัญชี ระบบตรวจสอบข้อมูลทั้งฝั่ง Client และ Server ก่อนเข้ารหัสรหัสผ่านและออก JWT Token ให้เข้าสู่ระบบโดยอัตโนมัติ ทั้งสองหน้ายึดหลัก Mobile-first Design เนื่องจากผู้สื่อข่าวมักตรวจสอบสื่อผ่านโทรศัพท์มือถือระหว่างปฏิบัติงานภาคสนาม ปุ่มและองค์ประกอบจึงจัดวางให้เข้าถึงง่ายด้วยนิ้วหัวแม่มือ พร้อมแสดงข้อความแจ้งเตือนแบบทันที เมื่อกรอกข้อมูลไม่ถูกต้อง เพื่อลดความหงุดหงิดและเพิ่มอัตราความสำเร็จในการสมัครสมาชิก

รูปที่ 3.13 หน้าสมัครสมาชิกและหน้าเข้าสู่ระบบของระบบ Lanner

3.11.4 หน้าอัปโหลดไฟล์เพื่อวิเคราะห์ (Analyze Media Page)

หน้านี้เป็นหน้าที่ผู้ใช้งานเข้าถึงเมื่อกดปุ่ม "เริ่มตรวจจับ" จากหน้าแรก ทำหน้าที่ทั้งรับไฟล์นำเข้าและแสดงผลลัพธ์ไว้ในหน้าเดียวกัน โดยไม่มีการเปลี่ยนหน้า (Page Navigation) ระหว่างขั้นตอนอัปโหลดกับขั้นตอนแสดงผล ซึ่งออกแบบตามหลัก Continuity of Context คือรักษาบริบทของผู้ใช้ไว้ในตำแหน่งเดิมตลอดกระบวนการ ลดความเสี่ยงที่ผู้ใช้จะสูญเสียโฟกัสหรือสับสนว่ากำลังอยู่ขั้นตอนใด ซึ่งเหมาะสมอย่างยิ่งกับผู้ใช้งานที่ต้องตรวจสอบไฟล์หลายรายการต่อเนื่องกันในเวลาทำงานจำกัด เช่น ผู้สื่อข่าวระหว่างปฏิบัติงาน ในขั้นตอนแรกหน้าแสดงพื้นที่ลากวางไฟล์ ที่ออกแบบด้วยเส้นขอบประ พร้อมมุมกรอบสี่มุมในลักษณะกรอบเล็ง (Viewfinder Frame) สื่อถึงการตรวจจับอย่างแม่นยำ พร้อมข้อความ "ลากและวางรูปภาพหรือวิดีโอ หรือคลิกเพื่อเลือกไฟล์จากอุปกรณ์ของคุณ" รองรับทั้ง Drag-and-Drop และการแตะเลือกไฟล์ตามหลัก Mobile-first Design ด้านล่างแสดงป้ายประเภทไฟล์ที่รองรับ และขนาดไฟล์สูงสุด เพื่อป้องกันข้อผิดพลาดล่วงหน้าตามหลัก Error Prevention

เมื่อผู้ใช้เลือกไฟล์ ระบบตรวจสอบเบื้องต้นฝั่ง Client ก่อนส่งไปยัง Backend และเปลี่ยนสถานะของ Drop Zone เป็นหน้าจอกำลังประมวลผล (Loading State) แสดง Progress Indicator เพื่อสื่อสารว่าระบบกำลังทำงานอยู่ ตามหลัก System Status Visibility ของ Nielsen's Usability Heuristics ป้องกันไม่ให้ผู้ใช้เข้าใจผิดว่าระบบค้างหรือไม่ตอบสนอง เมื่อประมวลผลเสร็จสิ้น พื้นที่เดิมจะเปลี่ยนเป็นส่วนแสดงผลลัพธ์ทันทีใน

ตำแหน่งเดียวกัน ประกอบด้วยป้ายผลลัพธ์ขนาดใหญ่ "REAL" สีเขียว หรือ "FAKE" สีแดง แถบ Confidence Score เป็นเปอร์เซ็นต์ ภาพตัวอย่างไฟล์พร้อมกรอบแสดงตำแหน่งใบหน้าที่ตรวจพบจาก RetinaFace และสำหรับไฟล์วิดีโอจะแสดงกราฟ Fake Score ตามเวลา เพิ่มเติม พร้อมปุ่ม "ตรวจสอบไฟล์ใหม่" ให้ผู้ใช้ย้อนกลับไปยัง Drop Zone เดิมเพื่อตรวจสอบไฟล์ถัดไปได้ทันทีโดยไม่ต้องโหลดหน้าใหม่

กรณีที่ผู้ใช้เข้าสู่ระบบเป็น Member ผลการตรวจสอบจะถูกบันทึกลงตาราง DetectionJob โดยอัตโนมัติ ส่วนกรณี Guest ระบบจะแสดงข้อความเชิญชวนให้สมัครสมาชิกเพื่อบันทึกประวัติไว้ใช้งานภายหลัง แทรกอยู่ในส่วนแสดงผลลัพธ์เดียวกันนี้

รูปที่ 3.14 หน้าอัปโหลดและแสดงผลการตรวจสอบ ของระบบ Lanner

3.11.5 หน้าประวัติการตรวจสอบ (History Page)

หน้าประวัติสำหรับ Member เท่านั้น ออกแบบในรูปแบบตาราง เพื่อรองรับหลัก Scannability คือให้ผู้ใช้กวาดสายตาดูรายการจำนวนมากได้อย่างรวดเร็ว โดยแสดงคอลัมน์ชื่อไฟล์ ประเภทไฟล์ ผลลัพธ์ (แสดงเป็นป้ายสีเขียว/แดงเช่นเดียวกับส่วนแสดงผลในหน้า Analyze Media) Confidence Score และวันที่ตรวจสอบ พร้อมตัวกรอง ตามช่วงวันที่และผลลัพธ์ เพื่อให้ผู้ใช้ค้นหารายการเฉพาะที่ต้องการได้โดยไม่ต้องไล่ดูทีละหน้า ผู้ใช้สามารถคลิกแต่ละแถวเพื่อดูรายละเอียดผลการตรวจสอบเดิมได้

ทั้งนี้หากไฟล์ต้นฉบับของรายการใดถูกลบไปแล้วตามนโยบายเก็บรักษา 30 วัน หน้านี้จะยังคงแสดงผลลัพธ์และ Confidence Score ไว้ตามปกติ พร้อมข้อความแจ้งเตือนสั้น ๆ ว่าไฟล์ต้นฉบับหมดอายุแล้ว แทนที่จะซ่อนรายการนั้นไปทั้งหมด เพื่อรักษาความสมบูรณ์ของประวัติการตรวจสอบไว้ให้ผู้ใช้อ้างอิงได้เสมอ สำหรับผู้ใช้ที่ยังไม่มีประวัติการตรวจสอบ หน้านี้จะแสดง Empty State พร้อมปุ่มลิงก์กลับไปหน้า Analyze Media เพื่อเริ่มตรวจสอบไฟล์แรก แทนที่จะปล่อยเป็นหน้าว่างเปล่าไม่มีคำแนะนำ

รูปที่ 3.15 หน้าประวัติการตรวจสอบ (History Page) ของระบบ Lanner

3.11.6 หน้า Admin Dashboard

หน้า Admin Dashboard ออกแบบสำหรับผู้ดูแลระบบโดยเฉพาะ แบ่งเนื้อหาออกเป็น 2 ส่วนหลักตามหลัก Progressive Disclosure เช่นเดียวกับส่วนแสดงผลในหน้า Analyze Media ส่วนแรกคือแถบสรุปสถิติภาพรวม แสดงจำนวนการตรวจสอบทั้งหมด สัดส่วนผล Real ต่อ Fake จำนวนผู้ใช้งานทั้งหมด และจำนวนรายการที่รอการยืนยัน เพื่อให้ Admin ประเมินสถานการณ์โดยรวมของระบบได้ในทันทีที่เข้าหน้านี้

ส่วนที่สองคือตารางรายการที่รอการยืนยัน ซึ่งกรองมาจากผลการตรวจสอบที่มีค่า Confidence Score อยู่ในช่วงก้ำกึ่ง (เช่น 40-60%) อันเป็นกรณีที่แบบจำลองไม่มั่นใจสูงพอ Admin สามารถตรวจสอบไฟล์ต้นฉบับประกอบการตัดสินใจ แล้วกดปุ่ม "ยืนยัน" หรือ "ปฏิเสธ" ผลลัพธ์นั้น ซึ่งทุกการกระทำจะถูกบันทึกลงตาราง SystemLog ด้วย Action ประเภท VERIFY_RESULT เพื่อใช้เป็นหลักฐานตรวจสอบย้อนหลังตามหัวข้อ 3.5 ถัดลงมาแสดงตารางประวัติการตรวจสอบทั้งระบบในรูปแบบเดียวกับหน้า History ของ Member แต่ครอบคลุมทุกผู้ใช้งาน พร้อมตัวกรองและช่องค้นหาสำหรับ Admin การแยกส่วน Pending Verification ออกจากตารางประวัติทั้งหมดอย่างชัดเจนเช่นนี้ เป็นการนำหลัก Actionable Notification มาใช้ คือเน้นให้ Admin เห็น

งานที่ต้องดำเนินการก่อนเป็นอันดับแรก แทนที่จะต้องไล่หารายการที่น่าสงสัยเองจากตารางขนาดใหญ่ปนกับรายการที่ไม่ต้องดำเนินการใดๆ

รูปที่ 3.16 หน้า Admin Dashboard ของระบบ Lanner

3.11.7หน้าบัญชีผู้ใช้งาน (Account Page)

หน้าบัญชีผู้ใช้งานสำหรับ Member เท่านั้น ออกแบบตามหลัก Self-service Account Management คือให้ผู้ใช้จัดการข้อมูลส่วนตัวของตนเองได้โดยไม่ต้องพึ่งพา Admin ลดภาระของผู้ดูแลระบบในงานสนับสนุนพื้นฐาน เนื้อหาแบ่งออกเป็น 2 ส่วนตามหลัก Grouping by Function ส่วนแรกคือข้อมูลบัญชี (Account Information) แสดงชื่อผู้ใช้ อีเมล บทบาท (Role) และวันที่สมัครสมาชิก โดยชื่อผู้ใช้และอีเมลสามารถแก้ไขได้ผ่านฟอร์มแก้ไขข้อมูล ซึ่งระบบตรวจสอบความถูกต้องทั้งฝั่ง Client และ Server เช่นเดียวกับหน้าสมัครสมาชิก

ส่วนที่สองคือการจัดการรหัสผ่าน (Change Password) แยกออกจากฟอร์มข้อมูลทั่วไปอย่างชัดเจน เนื่องจากเป็นการกระทำที่มีผลกระทบต่อความปลอดภัยของบัญชีโดยตรง ผู้ใช้ต้องกรอกรหัสผ่านเดิมเพื่อยืนยันตัวตนก่อนตั้งรหัสผ่านใหม่ ตามหลัก Re-authentication for Sensitive Actions เพื่อป้องกันกรณีที่มีผู้อื่นเข้าถึงอุปกรณ์ขณะที่ผู้ใช้ยังคงสถานะเข้าสู่ระบบอยู่ (Session Hijacking) เมื่อเปลี่ยนรหัสผ่านสำเร็จ ระบบจะบันทึกกิจกรรมลงตาราง SystemLog และแสดงข้อความยืนยันความสำเร็จให้ผู้ใช้ทราบทันที ด้านล่างสุดของหน้าแสดงปุ่ม "ออกจากระบบ" แยกด้วยระยะห่างและสีที่แตกต่างจากปุ่มอื่น เพื่อป้องกันการกดผิดพลาดโดยไม่ตั้งใจ (Accidental Action Prevention) ซึ่งเป็นตำแหน่งมาตรฐานที่ผู้ใช้คุ้นเคยจากระบบเว็บแอปพลิเคชันทั่วไป

รูปที่ 3.17 หน้าบัญชีผู้ใช้งาน (Account Page) ของระบบ Lanner

3.10 การออกแบบด้านความปลอดภัย (Security Design)

ระบบออกแบบด้านความปลอดภัยให้ครอบคลุมทั้งการยืนยันตัวตน การกำหนดสิทธิ์ การเข้ารหัสข้อมูล และการป้องกันการโจมตีเว็บทั่วไปตามมาตรฐาน OWASP Top 10 โดยยึดหลักการป้องกันในหลายชั้น เพื่อไม่ให้ระบบทั้งหมดล้มเหลวจากช่องโหว่จุดเดียว ทั้งนี้เนื่องจากระบบเปิดให้บุคคลทั่วไป เข้าถึงฟังก์ชันตรวจสอบ Deepfake ได้โดยไม่ต้องสมัครสมาชิก การออกแบบความปลอดภัยจึงต้องพิจารณาความเสี่ยงจากผู้ใช้งานที่ไม่ผ่านการยืนยันตัวตนควบคู่ไปกับการป้องกันข้อมูลของสมาชิกและผู้ดูแลระบบ เพื่อให้มั่นใจว่าทุกส่วนของระบบมีความปลอดภัยในระดับที่เหมาะสมกับบทบาทของผู้ใช้แต่ละประเภท

ตารางที่ 3.23 กลไกความปลอดภัยของระบบ

ลำดับ	กลไก	รายละเอียด
1	JWT Authentication	ใช้ JSON Web Token สำหรับยืนยันตัวตน กำหนดอายุ Token 24 ชั่วโมง จัดเก็บใน HttpOnly Cookie เพื่อป้องกัน Cross-Site Scripting (XSS)
2	Password Hashing	เข้ารหัสรหัสผ่านด้วย bcrypt (Salt Rounds = 12) ก่อนบันทึกลงฐานข้อมูล ไม่มีการจัดเก็บรหัสผ่านในรูปแบบ Plain Text

3	Role-Based Access Control (RBAC)	แยกสิทธิ์การเข้าถึงระหว่าง Guest, Member และ Admin โดยตรวจสอบบทบาทใน JWT Payload ก่อนทุก API Request
4	Re-authentication	บังคับให้กรอกรหัสผ่านเดิมซ้ำก่อนเปลี่ยนรหัสผ่านในหน้าบัญชีผู้ใช้งาน เพื่อป้องกัน Session Hijacking
5	File Type Validation	ตรวจสอบทั้ง MIME Type และ Magic Bytes ของไฟล์ที่อัปโหลด เพื่อป้องกันการอัปโหลดไฟล์ที่เป็นอันตราย
6	Input Validation	ตรวจสอบรูปแบบข้อมูลทุกรายการที่รับจากผู้ใช้ทั้งฝั่ง Frontend (Zod) และ Backend (Pydantic) เพื่อป้องกัน Injection Attack
7	HTTPS	บังคับใช้ HTTPS ในทุกการสื่อสารระหว่าง Client และ Server เพื่อป้องกันการดักฟังข้อมูล (Man-in-the-Middle)
8	CORS Policy	กำหนด CORS Policy ให้รับคำร้องขอเฉพาะจาก Domain ที่ได้รับอนุญาต เพื่อป้องกัน Cross-Origin Request ที่ไม่พึงประสงค์
9	Rate Limiting	จำกัดจำนวนคำร้องขอต่อ IP ที่ 20 ครั้งต่อนาทีสำหรับ Endpoint การตรวจสอบ เพื่อป้องกัน DDoS Attack
10	Pre-signed URL	ไฟล์ใน Cloudflare R2 จัดเก็บแบบ Private เข้าถึงผ่าน Pre-signed URL ที่มีอายุจำกัด ป้องกันการเข้าถึงไฟล์โดยไม่ได้รับอนุญาต
11	Audit Trail	บันทึกกิจกรรมสำคัญของผู้ใช้และ Admin ลงตาราง SystemLog เพื่อใช้ตรวจสอบย้อนหลังกรณีเกิดเหตุผิดปกติ

ในด้านการยืนยันตัวตน ระบบเลือกใช้ JWT ร่วมกับ HttpOnly Cookie แทนการจัดเก็บ Token ไว้ใน Local Storage เนื่องจาก Local Storage สามารถถูกเข้าถึงได้โดย JavaScript ฝั่ง Client ทำให้เสี่ยงต่อการถูกขโมย Token ผ่านช่องโหว่ XSS ในขณะที่ HttpOnly Cookie ป้องกันการเข้าถึงจากสคริปต์โดยตรง ส่วนการเข้ารหัสรหัสผ่านด้วย bcrypt ใช้ Salt Rounds ที่ 12 ซึ่งเป็นค่าที่สมดุลระหว่างความปลอดภัยและเวลาในการประมวลผล เนื่องจากค่าที่สูงเกินไปจะทำให้การเข้าสู่ระบบใช้เวลานานเกินความจำเป็น

ด้านการควบคุมสิทธิ์การเข้าถึง ระบบแบ่งบทบาทออกเป็น 3 ระดับตามที่กำหนดไว้ในตาราง User ได้แก่ Guest ที่สามารถตรวจสอบไฟล์ได้แต่ไม่สามารถดูประวัติ Member ที่สามารถดูและจัดการประวัติของตนเองได้ และ Admin ที่สามารถเข้าถึง Dashboard และยืนยันผลการตรวจสอบที่น่าสงสัยได้ โดยทุก API Endpoint ที่เกี่ยวข้องกับข้อมูลของ Member หรือฟังก์ชันของ Admin จะตรวจสอบบทบาทจาก JWT Payload ก่อนอนุญาตให้เข้าถึงเสมอ ป้องกันกรณีที่ผู้ใช้พยายามเข้าถึง Endpoint ที่เกินสิทธิ์ของตนโดยตรงผ่าน URL

ด้านการคุ้มครองข้อมูลส่วนบุคคล ระบบสอดคล้องกับพระราชบัญญัติคุ้มครองข้อมูลส่วนบุคคล พ.ศ. 2562 (PDPA) โดยกำหนดนโยบายเก็บรักษาไฟล์ต้นฉบับไว้เพียง 30 วันตามที่ระบุในหัวข้อ 3.8 และเก็บข้อมูลส่วนบุคคลเท่าที่จำเป็นต่อการใช้งานเท่านั้น (Data Minimization) ไม่มีการจัดเก็บข้อมูลอ่อนไหวอื่นนอกเหนือจากอีเมล ชื่อผู้ใช้ และรหัสผ่านที่เข้ารหัสแล้ว

3.10.1 มาตรฐาน OWASP Top 10:2025

เพื่อให้การออกแบบด้านความปลอดภัยของระบบมีมาตรฐานอ้างอิงที่ชัดเจนและเป็นที่ยอมรับในระดับสากล ผู้พัฒนาได้นำมาตรฐาน OWASP Top 10:2025 ซึ่งเป็นรายการความเสี่ยงด้านความปลอดภัยของเว็บแอปพลิเคชันที่พบบ่อยที่สุด 10 อันดับ จัดทำโดยองค์กร Open Web Application Security Project (OWASP) มาใช้เป็นกรอบในการตรวจสอบว่ากลไกความปลอดภัยที่ออกแบบไว้ในตารางที่ 3.23 ครอบคลุมความเสี่ยงแต่ละประเภทหรือไม่ ทั้งนี้ OWASP ได้ประกาศปรับปรุงรายการดังกล่าวเป็นฉบับปี 2025 เมื่อเดือนมกราคม พ.ศ. 2569 ซึ่งเป็นการปรับปรุงครั้งแรกนับตั้งแต่ฉบับปี 2021 โดยมีการเพิ่มหมวดใหม่ 2 หมวด จัดอันดับใหม่ 4 หมวด และเปลี่ยนชื่อ 3 หมวด

ตารางที่ 3.24 การสอดคล้องกับมาตรฐาน OWASP Top 10:2025

รหัส	หมวดความเสี่ยง	กลไกป้องกันในระบบ
A01	Broken Access Control	Role-Based Access Control ตรวจสอบบทบาทจาก JWT Payload ก่อนทุก API Request
A02	Security Misconfiguration	CORS Policy จำกัดเฉพาะ Domain ที่อนุญาต บังคับ HTTPS และเก็บค่าลับผ่าน Environment Variables
A03	Software Supply Chain Failures	ล็อกเวอร์ชัน Dependency ผ่าน package-lock.json และ requirements.txt
A04	Cryptographic Failures	เข้ารหัสรหัสผ่านด้วย bcrypt เข้ารหัสข้อมูลระหว่างส่งด้วย HTTPS และใช้ Pre-signed URL ที่มีอายุจำกัด
A05	Injection	Prisma ORM ป้องกัน SQL Injection ด้วย Parameterized Query ร่วมกับ Input Validation ผ่าน Zod และ Pydantic
A06	Insecure Design	ออกแบบ Re-authentication ก่อนเปลี่ยนรหัสผ่าน และ Rate Limiting ตั้งแต่ขั้นตอนออกแบบระบบ
A07	Authentication Failures	JWT Authentication ร่วมกับ HttpOnly Cookie ป้องกัน Token รั่วไหลผ่าน XSS
A08	Software and Data Integrity Failures	ตรวจสอบประเภทไฟล์ด้วย Magic Bytes ไม่ใช่แค่นามสกุลไฟล์
A09	Security Logging and Alerting Failures	บันทึกกิจกรรมสำคัญลงตาราง SystemLog ครอบคลุม LOGIN, LOGIN_FAILED, VERIFY_RESULT
A10	Mishandling of Exceptional Conditions	จัดการข้อผิดพลาดด้วย Try-Except คืนข้อความ Error ทั่วไป ไม่เปิดเผย Stack Trace

ตารางที่ 3.24 แสดงกลไกป้องกันที่ระบบมีอยู่ในระดับการออกแบบ สำหรับแต่ละหมวดความเสี่ยง โดยเฉพาะหมวด A09 ระบบมีเพียงกลไกการบันทึก Log ผ่านตาราง SystemLog แต่ยังไม่มีกลไกแจ้งเตือนอัตโนมัติเมื่อพบเหตุการณ์ผิดปกติ เช่น ความพยายามเข้าสู่ระบบผิดพลาดซ้ำหลายครั้งติดต่อกันในช่วงเวลาสั้น ซึ่งเป็นขอบเขตที่สามารถพัฒนาต่อยอดได้ในงานวิจัยหรือการพัฒนาระบบในอนาคต

3.11 แผนการทดสอบระบบ (System Testing Plan)

การทดสอบระบบเป็นกระบวนการสำคัญที่ช่วยยืนยันว่าระบบเว็บแอปพลิเคชันตรวจจับ Deepfake สามารถทำงานได้ถูกต้องตรงตามความต้องการที่วิเคราะห์และออกแบบไว้ แผนการทดสอบครอบคลุมทั้งประสิทธิภาพของแบบจำลอง AI ฟังก์ชันการทำงาน ความปลอดภัย และความพึงพอใจของผู้ใช้งานจริงจากสำนักข่าว Lanner News

3.11.1 กลยุทธ์การทดสอบ (Testing Strategy)

แผนการทดสอบใช้กลยุทธ์แบบ Bottom-Up Testing เริ่มทดสอบจากระดับ Unit ก่อน จากนั้นทดสอบการทำงานร่วมกัน ต่อด้วยการทดสอบระบบโดยรวม และสุดท้ายให้ผู้ใช้งานจริงทดสอบ เพื่อให้มั่นใจว่าระบบพร้อมใช้งานจริง

3.11.2 เครื่องมือที่ใช้ในการทดสอบ

ตารางที่ 3.25 เครื่องมือที่ใช้ในการทดสอบระบบ

ลำดับ	เครื่องมือ	ประเภทการทดสอบ	รายละเอียด
1	Jest + React Testing Library	Unit and Component Test	ทดสอบ React Component และ Business Logic ฝั่ง Frontend
2	pytest	Unit and Integration Test	Integration Test ทดสอบ FastAPI Backend และ Deepfake Detection Pipeline
3	Postman	API Test	ทดสอบ REST API Endpoints ยืนยัน HTTP Status Code และ Response Format
4	Playwright	End-to-End Test	จำลองการใช้งานจริงผ่านเบราว์เซอร์ ตั้งแต่อัปโหลดไฟล์จนแสดงผลลัพธ์
5	k6	Performance Test	ทดสอบจำนวนคำร้องขอพร้อมกันที่ระบบรองรับได้และ Response Time

3.11.3 การทดสอบประสิทธิภาพแบบจำลอง AI (Model Performance Testing)

การทดสอบนี้มีวัตถุประสงค์เพื่อยืนยันว่าแบบจำลอง XceptionNet ร่วมกับ Context Expansion สามารถจำแนก Real/Fake ได้แม่นยำเพียงพอ โดยทดสอบบน Testing Set ที่แบบจำลองไม่เคยพบมาก่อนตามหัวข้อ 3.3.4 ทั้งนี้เลือกพิจารณา Precision และ Recall ควบคู่กับ Accuracy เนื่องจากผลผิดพลาดทั้งสองทิศทางมีความเสี่ยงต่างกันในบริบทข่าว คือ False Positive อาจสร้างความเข้าใจผิดต่อข่าวจริง ขณะที่ False Negative อาจปล่อยให้ข้อมูลบิดเบือนหลุดรอดไปได้

ตารางที่ 3.26 กรณีทดสอบประสิทธิภาพแบบจำลอง AI

รหัส	กรณีทดสอบ	เกณฑ์การผ่าน
MT-01	Video-level Accuracy บน Testing Set	Accuracy ≥ 90%
MT-02	AUC-ROC Score	AUC ≥ 0.95
MT-03	Precision คลาส Fake	Precision ≥ 90%
MT-04	Recall คลาส Fake	Recall ≥ 90%
MT-05	Inference Time ต่อภาพ	≤ 500 มิลลิวินาที

3.11.4 การทดสอบเชิงฟังก์ชัน (Functional Testing)

ตารางที่ 3.27 กรณีทดสอบเชิงฟังก์ชัน

รหัส	กรณีทดสอบ	ผลลัพธ์ที่คาดหวัง
FT-01	สมัครสมาชิกด้วยอีเมลและชื่อผู้ใช้ใหม่	ระบบสร้างบัญชีและเข้าสู่ระบบอัตโนมัติ
FT-02	เข้าสู่ระบบด้วยข้อมูลถูกต้อง	ระบบออก JWT Token และแสดงหน้า Analyze Media
FT-03	อัปโหลดภาพและตรวจสอบ	แสดงผล Real/Fake พร้อม Confidence Score ในหน้าเดียวกัน
FT-04	อัปโหลดวิดีโอความยาวไม่เกิน 59 วินาที	แสดงผลระดับวิดีโอพร้อม Timeline Chart
FT-05	Member ดูประวัติการตรวจสอบ	แสดงรายการประวัติทั้งหมดของผู้ใช้นั้น
FT-06	อัปโหลดไฟล์ขนาดเกินกำหนด	แสดงข้อความแจ้งเตือนและไม่อัปโหลดไฟล์
FT-07	อัปโหลดไฟล์นามสกุลไม่รองรับ	ระบบปฏิเสธและแสดงข้อความแจ้งเตือน
FT-08	Admin ยืนยันผลที่รอตรวจสอบ	สถานะเปลี่ยนเป็นยืนยันแล้วและบันทึกลง SystemLog
FT-09	ผู้ใช้เปลี่ยนรหัสผ่านในหน้าบัญชี	ต้องกรอกรหัสผ่านเดิมก่อน จึงเปลี่ยนรหัสผ่านสำเร็จ

3.11.5 การทดสอบความปลอดภัย (Security Testing)

ตารางที่ 3.28 กรณีทดสอบความปลอดภัย

รหัส	กรณีทดสอบ	ผลลัพธ์ที่คาดหวัง
ST-01	เรียก API ที่ต้องการสิทธิ์โดยไม่มี JWT Token	คืน HTTP 401 Unauthorized
ST-02	Member พยายามเข้าถึง Admin Endpoint	คืน HTTP 403 Forbidden
ST-03	อัปโหลดไฟล์อันตรายที่เปลี่ยนนามสกุล	ระบบปฏิเสธไฟล์จากการตรวจสอบ Magic Bytes
ST-04	ส่ง SQL Injection ในช่อง Input	Prisma ORM ป้องกัน Query ไม่ถูก Execute
ST-05	ส่งคำร้องขอเกิน Rate Limit	คืน HTTP 429 Too Many Requests
ST-06	ตรวจสอบตาม OWASP Top 10:2025 Checklist	ผ่านทุกหมวด ไม่พบช่องโหว่ระดับ Critical/High

3.11.6 การทดสอบการยอมรับของผู้ใช้ (User Acceptance Testing: UAT)

การทดสอบ UAT ให้กลุ่มเป้าหมายซึ่งเป็นผู้สื่อข่าวและกองบรรณาธิการของสำนักข่าว Lanner News ทดลองใช้งานระบบจริงผ่าน Staging Environment ตามสถานการณ์ที่กำหนด แล้วประเมินความพึงพอใจแบบ Likert Scale ระดับ 1-5

ตารางที่ 3.29 สถานการณ์ทดสอบการยอมรับของผู้ใช้

รหัส	สถานการณ์ทดสอบ	เกณฑ์การผ่าน
UAT-01	สมัครสมาชิกและเข้าสู่ระบบครั้งแรก	เสร็จสิ้นภายใน 3 นาทีโดยไม่ต้องขอความช่วยเหลือ
UAT-02	อัปโหลดภาพข่าวที่สงสัยว่าเป็น Deepfake	เข้าใจผล Real/Fake และ Confidence Score โดยไม่ต้องอธิบายเพิ่ม
UAT-03	อัปโหลดวิดีโอข่าวและดู Timeline Chart	อ่านกราฟ Fake Score ตามเวลาได้อย่างถูกต้อง
UAT-04	ค้นหาและดูประวัติการตรวจสอบย้อนหลัง	ค้นหาและเปิดดูรายการเดิมได้สำเร็จ

UAT-05	ประเมินความพึงพอใจโดยรวม	คะแนน Likert Scale เฉลี่ย ≥ 4.0 / 5.0

3.11.7 เกณฑ์การผ่านการทดสอบ

ตารางที่ 3.30 สรุปเกณฑ์การผ่านการทดสอบระบบ

ประเภทการทดสอบ	จำนวนกรณีทดสอบ	เกณฑ์การผ่าน
Model Performance	5	Accuracy ≥ 90%, AUC ≥ 0.95
Functional Test	9	ผ่าน ≥ 95% ของกรณีทดสอบ
Security Test	6	ผ่าน 100% ไม่มีช่องโหว่ระดับ Critical/High
User Acceptance Test (UAT)	5	คะแนนความพึงพอใจเฉลี่ย ≥ 4.0 / 5.0
รวมทั้งหมด	25	ทุกประเภทต้องผ่านตามเกณฑ์ก่อนนำระบบไปใช้งานจริง

3.12 สรุปการออกแบบและพัฒนาระบบ

บทที่ 3 ได้นำเสนอกระบวนการออกแบบและพัฒนาเว็บแอปพลิเคชันตรวจจับ Deepfake อัจฉริยะเพื่อความมั่นคงปลอดภัยสารสนเทศ กรณีศึกษาสำนักข่าว Lanner News อย่างครบถ้วนในทุกมิติ เริ่มจากกระบวนการดำเนินงานตามวงจร Iterative Waterfall การวิเคราะห์กระแสข้อมูล 3 ระดับ (DFD Level 0-2) และการเตรียมชุดข้อมูล FaceForensics++ (C23) จำนวน 7,000 วิดีโอ ครอบคลุม 6 เทคนิค Deepfake โดยแบ่งชุดข้อมูลในอัตราส่วน 70:15:15 พร้อมเทคนิค Data Augmentation 8 รูปแบบ เพื่อเพิ่มความหลากหลายของข้อมูลสำหรับการฝึกสอน ในส่วนของวิธีการที่นำเสนอ ผู้พัฒนาใช้เทคนิค Context Expansion ขยายกรอบใบหน้าที่ตรวจจับได้จาก RetinaFace เพิ่มอีก 35% ร่วมกับแบบจำลอง XceptionNet ที่ผ่านการ Fine-tuning ด้วย Transfer Learning เพื่อเพิ่มความสามารถในการตรวจจับร่องรอยความผิดปกติบริเวณขอบใบหน้าในวิดีโอที่ผ่านการบีบอัด โดยประเมินประสิทธิภาพด้วยตัวชี้วัดมาตรฐาน 5 ตัว ได้แก่ Accuracy, Precision, Recall, F1-Score และ AUC-ROC

ด้านฐานข้อมูล ออกแบบโครงสร้าง PostgreSQL 4 ตารางหลัก ได้แก่ User, DetectionJob, FrameResult และ SystemLog พร้อมพจนานุกรมข้อมูลและตารางสรุปความสัมพันธ์ที่กำหนดไว้อย่างชัดเจน การออกแบบซอฟต์แวร์แบ่งเป็นโมดูลฝั่ง Frontend 5 โมดูลและฝั่ง Backend 6 โมดูล ตามหลัก Modular Design ส่วนการออกแบบเชิงพฤติกรรมครอบคลุมทั้ง Activity Diagram และ Sequence Diagram เพื่อแสดงลำดับการทำงานทั้งเชิงกระบวนการและการสื่อสารระหว่างส่วนประกอบของระบบ

ด้านการจัดเก็บข้อมูลและไฟล์ แยกจัดเก็บไฟล์สื่อใน Cloudflare R2 โดยกำหนดนโยบายเก็บรักษาไฟล์ต้นฉบับ 30 วัน ในขณะที่ผลการตรวจสอบยังคงถูกเก็บไว้อย่างถาวรในฐานข้อมูล เพื่อให้สมาชิกเรียกดูประวัติย้อนหลังได้เสมอ การออกแบบส่วนติดต่อผู้ใช้ครอบคลุม 7 หน้าหลัก ได้แก่ หน้าแรก หน้าวิธีการทำงาน หน้าเข้าสู่ระบบและสมัครสมาชิก หน้าอัปโหลดและแสดงผลการตรวจสอบ หน้าประวัติการตรวจสอบ หน้า Admin Dashboard และหน้าบัญชีผู้ใช้งาน โดยอ้างอิงหลักการ UX/UI ที่เหมาะสมกับกลุ่มเป้าหมายซึ่งเป็นผู้สื่อข่าวและกองบรรณาธิการในแต่ละหน้า

ด้านความปลอดภัย ระบบใช้กลไก JWT, bcrypt, RBAC, Input Validation และ Rate Limiting ร่วมกับการตรวจสอบความสอดคล้องกับมาตรฐาน OWASP Top 10:2025 ครบทั้ง 10 หมวด พร้อมยอมรับข้อจำกัดที่ยังพัฒนาต่อได้ในด้าน Alerting สุดท้าย แผนการทดสอบกำหนดกลยุทธ์ Bottom-Up

Testing พร้อมเครื่องมือทดสอบที่เหมาะสมกับแต่ละระดับ รวมกรณีทดสอบทั้งหมด 25 กรณี ครอบคลุม Model Performance, Functional, Security และ User Acceptance Testing โดยให้ผู้สื่อข่าวและกองบรรณาธิการของ Lanner News ร่วมทดสอบจริง เพื่อยืนยันว่าระบบมีคุณภาพและพร้อมนำไปใช้งานจริงในบริบทของงานสื่อสารมวลชนและการตรวจสอบข้อเท็จจริงในสังคมดิจิทัล